



UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA

Scuola di Scienze

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di laurea in Informatica

Comparative Study of Network Emulators: A Detailed Analysis of Mininet and Kathara

Relatore: Prof. Savi Marco

Correlatore: Dott. Talpini Jacopo

Relazione della prova finale di:

Broglia Matteo

Matricola 899562

Anno Accademico 2024 - 2025

To Professor Savi and Doctor Talpini, I extend my heartfelt gratitude for your support and constant availability throughout these months. Your dedication to our daily work has been invaluable, and it has been an absolute pleasure collaborating with you. Your evident passion for your work has driven me to work to the best of my abilities and to explore with immense curiosity, going beyond what was covered in the thesis.

To my parents and grandparents, Carla and Tino, for their constant support, for having my back in every choice I made, celebrating each success, and offering strength during the most challenging moments. Your sacrifices, wisdom, and encouragement have shaped me into who I am today. Thank you for believing in me, your love and guidance have been invaluable.

To Federico, Gabriele, Lorenzo, Riccardo, and the other friends who have been by my side through this journey. Your companionship, encouragement, and shared moments have made this experience unforgettable. I look forward to continuing this journey alongside you as we embark on further studies together, supporting each other every step of the way.

Abstract

Network emulation is essential for testing, research, and education, providing controlled environments that mimic real-world network behaviour with high fidelity. This thesis presents a comprehensive comparison of two leading network emulators, Mininet and Kathara, analysing their architectural designs, functional capabilities, limitations, and practical applications. Through rigorous theoretical analysis and empirical benchmarking, it assess performance across critical metrics, encompassing resource utilization, scalability, deployment complexity, and extensibility.

The findings reveal distinct trade-offs between extensibility, simplicity, and flexibility. Mininet demonstrates superior resource efficiency due to its lightweight architecture utilizing Linux namespaces and a shared kernel, making it ideal for large-scale topologies on limited hardware. However, this comes at the cost of reduced isolation between network elements. In contrast, Kathara’s Docker-based containerization provides stronger isolation and greater platform compatibility (Windows, macOS, Linux) but consumes significantly more CPU and memory resources, particularly evident during initialization and shutdown phases.

For educational contexts, Kathara’s container-based isolation and detailed documentation offer a clearer separation of concerns, while its Lab Generator tool simplifies complex setups. For research applications, Mininet’s extensive community support and numerous extensions, like IPMininet and Mininet-WiFi provide specialized functionality for various networking paradigms.

The thesis also examines community-developed extensions for both platforms, highlighting Containernet as a potential integration pathway combining Mininet’s efficiency with Docker’s isolation capabilities. Practical guidelines are provided for selecting the appropriate emulation tool based on specific requirements, actionable implementation recommendations are offered, and future research directions are explored to address the limitations identified in this comparative analysis.

Contents

1	Introduction	1
1.1	Definition and Purpose of Network Emulation	1
1.2	Common Use Cases for Network Emulators	1
1.3	Comparison with Network Simulators	3
2	Mininet: A Detailed Analysis	5
2.1	Overview of Mininet	5
2.2	Architecture and Components	5
2.3	Use Cases and Applications	6
3	Kathara: A Detailed Analysis	9
3.1	Overview of Kathara	9
3.2	Evolution from Netkit to Kathara	9
3.2.1	Differences between Netkit and Old Kathara	9
3.2.2	Differences between Old Kathara and New Kathara	10
3.3	Architecture and Components	11
3.3.1	Core Concepts	11
3.3.2	Main Components	11
3.4	Use Cases and Applications	12
4	Non-Functional Benchmarking	13
4.1	Benchmark Design and Metrics	13
4.2	Test Scenarios and Setup	13
4.2.1	Simple Network with Two Hosts	14
4.2.2	Static Routing	15
4.2.3	Client - Server	16
4.2.4	Load Balancing	17
4.2.5	RIP	19
4.2.6	OSPF	21
4.2.7	BGP	23
4.2.8	Multiple Solutions: BGP, OSPF, RIP and WebServer	25
4.2.9	DNS	28
5	Non-Functional Analysis Results	37
5.1	Performance Test Results	37
5.1.1	Scenarios Structure	38
5.1.2	Hardware Specifications	38
5.1.3	Simple Network with Two Hosts	39
5.1.4	Static Routing	40

5.1.5	Client Server	41
5.1.6	Load Balancing	42
5.1.7	RIP	43
5.1.8	OSPF	44
5.1.9	BGP	45
5.1.10	Multiple Solutions: BGP, OSPF, RIP and WebServer	46
5.1.11	DNS	47
5.2	Comparative Analysis Based on Benchmarking Data	48
6	Functional Analysis	51
6.1	Architectural Comparison	51
6.2	Flexibility and Extensibility	52
6.2.1	Mininet	52
6.2.2	Kathara	53
6.3	Ease of Use and Documentation	55
6.3.1	Mininet	55
6.3.2	Kathara	56
6.3.3	Summary	57
7	Conclusion and Future Work	59
7.1	Recommendations	59
7.1.1	Mininet	59
7.1.2	Kathara	60
7.2	Future Work	61
	Afterword	63
	Bibliography	68

Chapter 1

Introduction

All the material I created and used for this thesis is available on GitHub. [1]

1.1 Definition and Purpose of Network Emulation

Network emulation is a process that creates a controlled environment that replicates the behaviour of a computer network without requiring the actual physical network hardware. Network emulators build functional replicas of network systems that behave like real networks, so that actual applications and protocols can be run within them.

The fundamental structure of network emulators involves the creation of virtual nodes (e.g., routers, switches, computers) interconnected by virtual links with configurable properties. They are implemented in software but possess the same characteristics as the hardware counterparts. Network emulators emerged as a response to the need to simulate network applications, protocols, and configurations in a testing environment before running them in production.

They were developed to address several limitations [2, 3]:

- **Cost Constraints:** Building multiple physical network testbeds for experimentation is highly expensive.
- **Scale Limitations:** Physical testbeds can rarely match the scale of modern networks.
- **Reproducibility:** Real networks are subject to unpredictable external factors, making experimental reproducibility difficult.
- **Safety:** It is dangerous to try potentially disruptive configurations or security exploits on live networks.

1.2 Common Use Cases for Network Emulators

According to the literature [4, 5, 6, 7, 8], network emulators are widely used in various fields, including education, research, development, and testing. They provide a versatile platform for simulating network conditions and behaviours, enabling users to experiment with different scenarios without the risks associated with real networks. The following sections outline the primary use cases for network emulators:

Educational Applications Network emulators serve as teaching tools that allow students to:

- Gain hands-on experience without expensive hardware
- Experiment with network protocols
- Develop troubleshooting skills
- Understand networking concepts through practical engagement

Testing and Development For developers and engineers, emulators offer:

- Application testing under various network conditions
- Protocol validation across different topologies
- Distributed application behaviour testing
- Pre-deployment validation of network changes

Research and Innovation Researchers use emulators to:

- Develop new networking protocols
- Test routing algorithms
- Evaluate security mechanisms
- Explore scalability challenges

Network Design and Planning Network architects use emulators to:

- Validate designs before physical implementation
- Optimize traffic engineering policies
- Test failover mechanisms
- Evaluate the impact of new services

Security Analysis Cybersecurity professionals employ network emulators to:

- Create environments for red team/blue team exercises
- Test intrusion detection systems
- Analyse attack vectors
- Train security personnel

1.3 Comparison with Network Simulators

Network simulation and emulation are valuable tools for studying computer networks, each with distinct purposes and methods. Understanding their differences helps in choosing the right approach for a given task.

Key Differences

- **Execution of Protocols:** Emulators execute real network protocol implementations, enabling actual applications such as web browsers, to operate within the emulated environment as they would in a live network. Simulators, by contrast, rely on abstract, algorithmic representations of protocols, simulating their behaviour without running genuine protocol stacks.
- **Realism vs. Scalability:** Emulators deliver high realism by replicating authentic network conditions, including packet loss, latency, and bandwidth constraints, and they support live traffic generation. This makes them ideal for testing real-world performance. Simulators prioritize scalability, enabling the simulation of large networks with many nodes, but sacrifice fidelity and real-time interaction.
- **Performance and Resource Needs:** Emulators demand lots of computation as they emulate full network stacks and process real packets in real time. For instance, emulation of a complex network with many virtual routers and switches can strain CPU and memory. Simulators, while being less resource-intensive, sacrifice speed of execution and accuracy to facilitate large-scale modelling on modest hardware.
- **Time Management:** Simulators use a step-by-step approach, where time moves forward based on specific events like sending a packet or updating a route. This makes it easier and faster to study how a network behaves over a long period. Emulators, on the other hand, run in real time, mimicking the actual timing of a physical network. That is very important when testing things like delay-sensitive apps.
- **Interaction with External Systems:** Since emulators work with real protocols, they can easily connect to actual hardware, like a real server or client and blend with outside networks, bridging the emulated and real worlds. Simulators, designed as closed systems, lack this capability, focusing instead on theoretical or isolated studies.

Complementary Use Rather than being mutually exclusive, emulation and simulation often complement each other in a hybrid workflow. For example, simulations can be used in the early stages of network design to explore a broad range of topologies and theoretical scenarios, identifying promising configurations. Emulation can then refine these findings by testing real implementations under realistic conditions before final deployment. This combined approach maximizes both scalability and accuracy, offering a robust toolkit for network research, development, and training. [3, 9, 10, 11]

Chapter 2

Mininet: A Detailed Analysis

2.1 Overview of Mininet

Mininet [12] represents a breakthrough in network emulation technology, first released on GitHub [13] on December 6, 2009, based on Bob Lantz’s original model [3]. Just a few years later, in 2012, the team officially launched version 2.0. This remarkable software lets users build realistic virtual networks that run actual kernel code, switch implementations, and network applications, all on a single computer.

Before Mininet introduction, researchers studying networks faced substantial challenges: expensive hardware requirements, limited ability to scale experiments, and highly complex setup procedures for testing network configurations. The costs alone put serious network research beyond the reach of many institutions and individual researchers.

The creators developed Mininet specifically to tackle these obstacles. They thought of a virtual environment combining simulation benefits (affordability and easy reconfiguration) with the crucial advantage of running real network code. As Software-Defined Networking research took off, Mininet quickly became the main tool for anyone wanting to prototype, test, or validate Software-Defined Network (SDN) applications and controllers without massive infrastructure investments.

2.2 Architecture and Components

Mininet runs exclusively on Linux systems and relies on several technologies to create its virtual network environment [3]:

- **Links:** A virtual Ethernet pair, or veth pair, acts like a wire connecting two virtual interfaces; packets sent through one interface are delivered to the other, and each interface appears as a fully functional Ethernet port to all system and application software. Veth pairs may be attached to virtual switches such as the Linux bridge or an OpenFlow software switch.
- **Hosts:** Network namespaces serve as containers for network state. They provide processes with exclusive ownership of interfaces, ports, and routing tables (e.g., ARP and IP). A host in Mininet is simply a shell process moved into its own network namespace with the `unshare(CLONE_NEWNET)` system call. Each host has its own virtual Ethernet interfaces and a pipe to a parent Mininet process, `mn`, which sends commands and monitors output.

- **Switches:** Software OpenFlow switches provide the same packet delivery semantics that would be provided by a hardware switch. Both user-space and kernel-space switches are available.
- **Controllers:** Controllers can be anywhere on the real or simulated network, as long as the machine on which the switches are running has IP-level connectivity to the controller. When Mininet is running in a virtual machine (VM), the controller can operate inside the VM, natively on the host machine, or in the cloud.

The entire architecture follows a clear hierarchical organization:

- A Mininet instance encloses the complete virtual network.
- Nodes (including hosts, switches, and controllers) function as the network devices.
- Links provide the connections between these various nodes.

The host operating system’s kernel handles all the underlying network virtualization infrastructure, making this approach remarkably efficient with system resources.

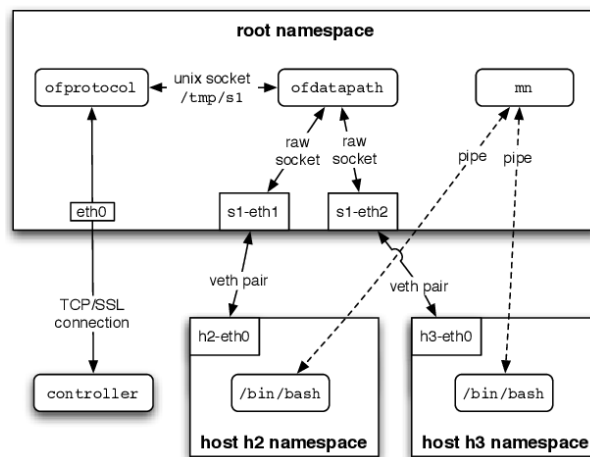


Figure 2.1: Mininet Architecture [3]

2.3 Use Cases and Applications

Mininet is being used across numerous networking domains:

- **Academic Research:** Mininet is extensively utilized by researchers and research labs to study SDN, Network Function Virtualization, and even the deployment of new protocols at no-hardware-cost installations. [14]
- **Education and Training:** Professors and teachers use Mininet as an education platform, and students can experiment with network concepts and architectures directly on their laptops. [15]
- **SDN Controller Development:** Teams working on controllers like ONOS, OpenDaylight, and Ryu usually validate their implementation with Mininet environments. [16]

- **Network Application Prototyping:** Companies develop and test network applications in Mininet prior to deployment on production networks. [14]
- **Protocol Testing:** Developers can test protocol modifications securely in controlled Mininet environments. [17]
- **Security Research:** Security teams use Mininet to study vulnerabilities, simulate network attacks, and develop defensive strategies. [18]
- **Data Center Planning:** Architects model data centre network designs in Mininet before committing to physical implementation. [19]
- **Telecommunication Network Planning:** Service providers use Mininet to evaluate proposed changes and their potential impacts on larger networks. [20]
- **Internet-Scale Research:** Researchers studying routing algorithms, congestion control, and other large-network challenges benefit from Mininet's scaling capabilities. [21]

As networking evolves, Mininet continues to grow alongside it. Developers regularly release extensions and modifications to address emerging requirements and use cases in this rapidly changing field.

Chapter 3

Kathara: A Detailed Analysis

3.1 Overview of Kathara

Kathara [22, 23, 24] was first developed as an open-source network emulation system and launched on GitHub [25] on March 26, 2017. The project is a spin-off of Netkit [26], it addresses numerous limitations of its predecessor while maintaining support for existing labs. The name "Kathara" is a reference to the Sanskrit word "*kathārā*", etymologically meaning "clean" or "pure," reflecting the goal of creating a more powerful, cleaner network emulation system.

The project began at Roma Tre University. The major motivation behind its development was to formulate an efficient and scalable solution for network design, testing, and analysis of network configurations in a simulated, controlled, and virtualized platform. In contrast to other network simulators that rely heavily on abstract models, Kathara is directed toward a more utilitarian solution focused on container-based virtualization. This approach enables users to test actual network scenarios in highly detailed environments, making it an excellent teaching tool and platform for complex studies.

3.2 Evolution from Netkit to Kathara

The evolution of Kathara has been documented in research by its developers [24], which outlines the transition from Netkit to Kathara, the key differences between versions, and the system's architectural design.

1. **Netkit:** The original network emulation tool that utilized User-Mode Linux (UML) virtual machines.
2. **Old Kathara (up to version 0.36.1):** The first release of Kathara that introduced containerization using Docker [27, 23].
3. **New Kathara (version 2.0 onward):** A complete rewrite of the codebase introducing multithreading and support for multiple virtualization technologies. [24]

3.2.1 Differences between Netkit and Old Kathara

The transition from Netkit to Old Kathara brought several significant changes:

- **Virtualization Technology:** While Netkit used UML virtual machines, Old Kathara implemented devices using Docker containers, reducing overhead by eliminating the need for multiple kernels.
- **Filesystem Management:** Docker images simplified the management of different filesystem images, allowing devices to easily use different images within the same network scenario.
- **Cross-Platform Support:** Unlike Netkit, which was limited to Linux platforms, Old Kathara could run on all major operating systems (Windows, macOS, and Linux) due to its Docker foundation.
- **Performance Limitations:** Old Kathara was single-threaded, whereas Netkit utilized parallel execution for starting up and shutting down network scenarios.
- **Security Considerations:** Netkit ran its virtual machines in user space, making it resilient to privilege escalation attacks. In contrast, Old Kathara required root privileges to run containers and started its containers with the `privileged` flag, making it more vulnerable to such attacks.
- **Collision Domain Implementation:** Old Kathara implemented collision domains using Docker bridge networks, which required patching to behave like hubs rather than switches. Additionally, Docker networks required an IP subnet, which did not align with the pure definition of a collision domain.

3.2.2 Differences between Old Kathara and New Kathara

The evolution from Old Kathara to New Kathara (version 2.0 and beyond) brought several enhancements:

- **Multithreading Support:** New Kathara introduced multithreading for starting up and shutting down network scenarios, significantly improving performance.
- **Multiple Virtualization Technologies** The codebase was completely rewritten to support multiple managers, overcoming the limitation of Old Kathara, which only supported Docker. New Kathara implements both a Docker Manager and a Kubernetes Manager (called Megalos [28, 29]).
- **Improved Collision Domain Implementation:** Instead of patching Docker bridge networks, New Kathara developed a custom Docker Network Plug-in specifically designed to implement Kathara collision domains. This eliminated the performance loss associated with applying patches for each collision domain and created pure Layer 2 LANs without requiring IP subnets.
- **Enhanced Security:** New Kathara eliminated the requirement for the `privileged` flag when starting containers, reducing the risk of privilege escalation attacks, though the option remains available if needed.

3.3 Architecture and Components

According to the developers, Kathara’s architecture is built around three core concepts and three main components that form the foundation of its framework.

3.3.1 Core Concepts

- **Device:** A virtual entity emulating a real network device. It consists of one or more virtual network interfaces, a virtual CPU with RAM, and a virtual disk. Kathara devices can emulate various network elements such as routers, DNS servers, web servers, or any other network device.
- **Collision Domain:** A virtual Layer 2 LAN that interconnects devices. It functions like a physical link between device interfaces, behaving essentially as a network hub. Each collision domain forwards all packets received from one interface to all other interfaces connected to that same domain without modifying the packets.
- **Network Scenario:** A complete representation of a network topology consisting of multiple devices connected through collision domains. In Kathara, a network scenario is represented as a directory containing:
 - A file describing the network topology
 - Individual files and folders for each device containing their specific configurations

3.3.2 Main Components

The Kathara architecture (illustrated in Figure 3.1) consists of three main components as described by its developers:

- **CLI (Command Line Interface):** Provides a set of shell commands that allow users to interact with Kathara modules. These commands are categorized into:
 - **v-prefixed** commands: Used to configure, start up, and shut down a single network device
 - **l-prefixed** commands: Used to configure, start up, and shut down a whole network scenario
 - **General commands:** General purpose commands for system settings, getting information about running devices, and performing global actions on devices and scenarios
- **Parser:** Responsible for converting the network scenario into Kathara domain objects that are agnostic to the underlying virtualization technology. This component serves as an intermediary between the network scenario description and the managers, converting the chosen format into domain objects. The parser’s design allows network scenarios to be described in various formats, as long as the parser logic is adapted accordingly.

- **Manager:** Interacts with a specific virtualization engine by converting a network scenario (represented with domain objects obtained from the Parser) into the virtualization engine's language. Managers implement a common interface, allowing Kathara to support multiple virtualization technologies. This design significantly increases the system's modifiability and flexibility.

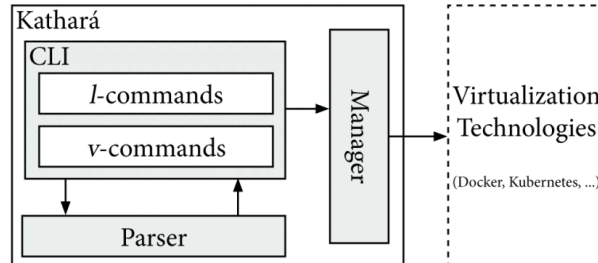


Figure 3.1: Kathara Architecture [23]

3.4 Use Cases and Applications

The multi-purpose capability of Kathara makes it a valuable asset in a wide range of use cases, across academic, professional, and research purposes. Its primary use cases include:

- **Education and Training:** University learning and training programs extensively use Kathara in teaching network fundamentals. Students can test IP routing, subnetting, and protocol configuration concepts in a secure, repeatable network. The inclusion of packet sniffers allows hands-on learning by permitting real-time traffic analysis.
- **Network Design and Testing:** Network experts use Kathara for network designs and testing for deployment. Through multi-device emulation, they can test network configurations, identify bottlenecks, and test protocol functionality without risking production networks.
- **Research and Development:** Developers use Kathara in exploring new paradigms for networks, such as Software Defined Network (SDN) or Internet of Things (IoT) implementations. The use of containers allows for rapid prototyping and iterative testing of custom protocols or custom network designs.
- **Cybersecurity Training:** The capability of Kathara in modelling realistic networks makes it best for cybersecurity exercises, such as penetrations, defence against simulated attacks, and network vulnerability probing. The inclusion of hosts supporting a web interface and packet sniffers enhance real-time analysis of network security and functionality in a live, online situation.

In summary, Kathara is a powerful, yet accessible, network emulation solution. Its lightweight architecture, built on Docker, combined with its practical add-ons and flexible configuration options, make it a perfect solution for anyone who wishes to learn about network environments through emulation. In corporate test labs, schools, and research establishments, it continues to demonstrate its usefulness and applicability in the ever-changing field of networking. [30, 31]

Chapter 4

Non-Functional Benchmarking

4.1 Benchmark Design and Metrics

This section outlines the benchmarks used to evaluate the performance of different network emulators and setups. Comparison Metrics include:

- **Memory Usage:** Evaluates the percentage of memory consumed by the emulators during different test scenarios.
- **CPU Utilization:** Measures the percentage of CPU resources utilized during emulation.

The benchmark is structured to ensure repeatability and reproducibility. Each test scenario is executed multiple times under controlled conditions to collect accurate and reliable data. Metrics are collected using system tools like `ps` and `aux`.

4.2 Test Scenarios and Setup

The test cases illustrated in this section are aimed at a wide variety of functionalities and protocols. Each case has been crafted with specific considerations to assess scalability, supported protocols, and resource efficiency. Every scenario is implemented using both Mininet and Kathara.

The lines of code and configuration files that define the topology are not reported here, as they do not relate directly to the protocols and services being evaluated. Only script fragments and relevant configuration details are shown in order to highlight important functional aspects.

4.2.1 Simple Network with Two Hosts

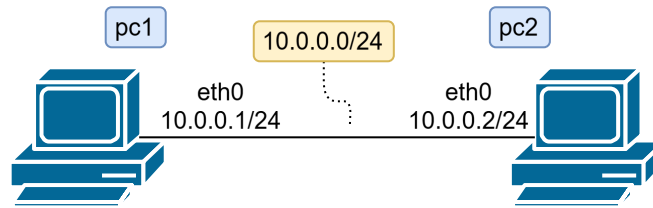


Figure 4.1: Basic two-host Network Topology

This scenario features two hosts on the same network. The purpose is to measure basic performance with each emulator, so this will be the baseline for further, more complex tests. It facilitates an initial comparison of Mininet and Kathara in terms of setup simplicity and resource utilization.

This section provides a complete introductory reference by including all the necessary files, such as topology definitions, configuration files, and relevant scripts, in order to clearly understand how both emulators define and manage even the simplest network setups.

Mininet

```

twoHosts.py

#!/usr/bin/python3

from mininet.net import
↳ Mininet

if __name__ == "__main__":
    net = Mininet()

    pc1 = net.addHost("pc1")
    pc2 = net.addHost("pc2")

    net.addLink(pc1, pc2)

    net.start()

    pc1.setIP("10.0.0.1/24",
↳ intf="pc1-eth0")
    pc2.setIP("10.0.0.2/24",
↳ intf="pc2-eth0")

    net.stop()
  
```

Kathara

The Kathara lab folder contains the following files: *lab.conf*, *pc1.startup*, and *pc2.startup*.

lab.conf

```

pc1[0]=A
pc1[image]=kathara/base
pc2[0]=A
pc2[image]=kathara/base
  
```

pc1.startup

```

ip address add 10.0.0.1/24 dev eth0
  
```

pc2.startup

```

ip address add 10.0.0.2/24 dev eth0
  
```

4.2.2 Static Routing

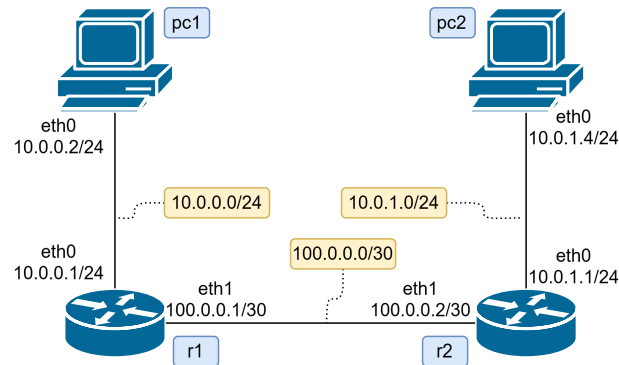


Figure 4.2: Static Routing Scenario

This scenario involves two hosts on separate networks communicating via two routers using static routing. The test aims to demonstrate how Mininet and Kathara manage basic inter-network communication and routing configuration.

Mininet

StaticRouting.py

```
pc1.cmd("ip route add default via 10.0.0.1")
pc2.cmd("ip route add default via 10.0.1.1")

r1.cmd("ip route add 10.0.1.0/24 via 100.0.0.2")
r1.cmd("sysctl -w net.ipv4.ip_forward=1")

r2.cmd("ip route add 10.0.0.0/24 via 100.0.0.1")
r2.cmd("sysctl -w net.ipv4.ip_forward=1")
```

Kathara

pc1.startup

```
ip route add default via 10.0.0.1 dev eth0
```

pc2.startup

```
ip route add default via 10.0.1.1 dev eth0
```

r1.startup

```
ip route add 10.0.1.0/24 via 100.0.0.2 dev eth1
```

r2.startup

```
ip route add 10.0.0.0/24 via 100.0.0.1 dev eth1
```

4.2.3 Client - Server

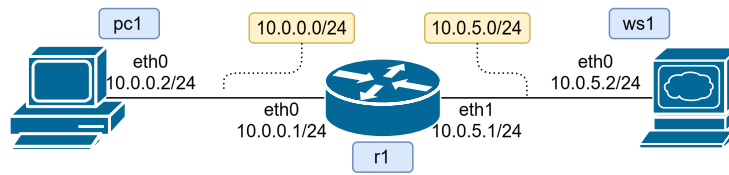


Figure 4.3: Client - Server Scenario

In this section there are two hosts, one acting as a Client (left) and the other one acting as a Web Server (right). The topology is kept as simple as possible using only one router to emulate communication between two different networks.

The objective is to illustrate the configuration and access of a web server, as well as the traffic it generates.

The web server contains a simple file named *index.html* with the following content:

```
index.html
<html>
  <body>
    This is the Web Server
  </body>
</html>
```

Mininet

```
webServer.py

pc1.cmd("ip route add default via 10.0.0.1")

ws1.cmd("ip route add default via 10.0.5.1")
ws1.cmd("python3 -m http.server 80 &")
```

Kathara

The lab is made of one folder (*ws1*) representing the Web Server, it contains the *index.html* file with the following path *var/www/html/index.html*. The Web Server is running apache.

```
pc1.startup

ip route add default via 10.0.0.1 dev eth0
```

```
ws1.startup

ip route add default via 10.0.5.1 dev eth0
systemctl start apache2
```

4.2.4 Load Balancing

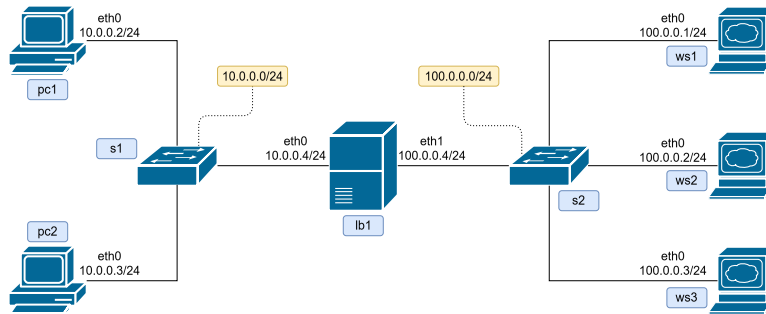


Figure 4.4: Load Balancing Scenario with three Web Servers and random traffic distribution

Two computers, acting as clients, request services from a server farm, with a load balancer (*lb1*) randomly distributing traffic to a specific server within the farm. Using *iptables*, filters are defined to distribute requests across the servers. In this scenario, these filters ensure equal probability of forwarding requests to each of the three available servers. Each server hosts a unique *index.html* file, designed solely to identify which server responds to a given request.

Mininet

It is important to write *iptables* commands on the same line otherwise they will not be executed correctly

loadBalancer.py

```
ws1.cmd("ip route add default via 100.0.0.4")
ws1.cmd("cd /var/www/html/ws1 && python3 -m http.server 80 &")

ws2.cmd("ip route add default via 100.0.0.4")
ws2.cmd("cd /var/www/html/ws2 && python3 -m http.server 80 &")

ws3.cmd("ip route add default via 100.0.0.4")
ws3.cmd("cd /var/www/html/ws3 && python3 -m http.server 80 &")

lb1.cmd("iptables-legacy --table nat --append PREROUTING --destination
↪ 10.0.0.4 -p tcp --dport 80 --match statistic --mode random --probability
↪ 0.33 --jump DNAT --to-destination 100.0.0.1:80")

lb1.cmd("iptables-legacy --table nat --append PREROUTING --destination
↪ 10.0.0.4 -p tcp --dport 80 --match statistic --mode random --probability
↪ 0.5 --jump DNAT --to-destination 100.0.0.2:80")

lb1.cmd("iptables-legacy --table nat --append PREROUTING --destination
↪ 10.0.0.4 -p tcp --dport 80 --jump DNAT --to-destination 100.0.0.3:80")
```

Kathara

The lab consists of three folders, one for each Web Server, containing different *index.html* files, a *lab.conf* file and a *.startup* file for each device.

ws1.startup

```
ip route add default via 100.0.0.4 dev eth0
systemctl start apache2
```

ws2.startup

```
ip route add default via 100.0.0.4 dev eth0
systemctl start apache2
```

ws3.startup

```
ip route add default via 100.0.0.4 dev eth0
systemctl start apache2
```

lb1.startup

```
iptables-legacy --table nat --append PREROUTING --destination 10.0.0.4 -p tcp
↪ --dport 80 --match statistic --mode random --probability 0.33 --jump DNAT
↪ --to-destination 100.0.0.1:80

iptables-legacy --table nat --append PREROUTING --destination 10.0.0.4 -p tcp
↪ --dport 80 --match statistic --mode random --probability 0.5 --jump DNAT
↪ --to-destination 100.0.0.2:80

iptables-legacy --table nat --append PREROUTING --destination 10.0.0.4 -p tcp
↪ --dport 80 --jump DNAT --to-destination 100.0.0.3:80
```

4.2.5 RIP

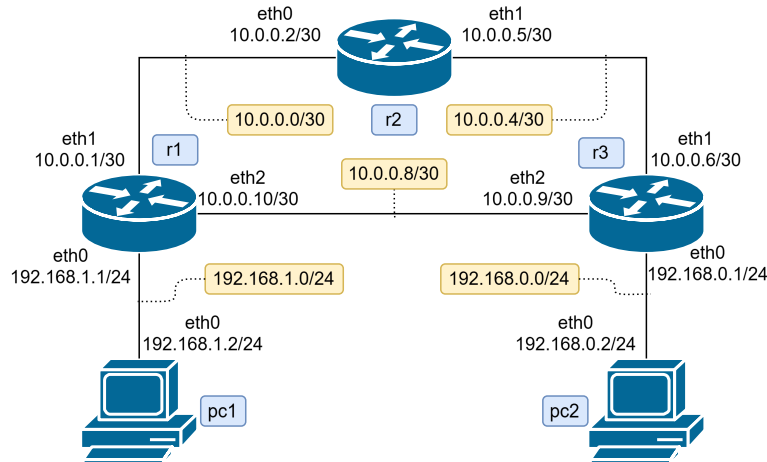


Figure 4.5: RIP Protocol Scenario

In this scenario, the two hosts communicate through three interconnected routers using the Routing Information Protocol (RIP). The objective is to evaluate how Mininet and Kathara handle dynamic routing and overall network performance under RIP. This is the first scenario where FRRouting (FRR) is introduced, providing a comparison of the key setup differences between Mininet and Kathara. The implementation of FRR highlights how each emulator manages routing daemons, configuration files and network isolation while also highlighting the key differences between a Non-Containerized system and a Containerized one.

Mininet

To get `vttysh` and `ripd` (the RIP daemon) working properly, I had to make some edits to their configuration files.

First, I had to make sure `frr` was properly installed and edit the file `/etc/frr/daemons` that controls what routing daemons are enabled in the FRR suite. `ripd` is off by default, so I changed the line that reads `ripd=no` to `ripd=yes` to enable the RIP daemon. That is important because without it RIP would not run on the routers in the Mininet network. Then, to make everything work properly with multiple routers in the Mininet topology, I needed to set up a dedicated configuration for each router. On my local machine, under the FRR directory (`/etc/frr`), I created a subfolder for each router to keep the configurations isolated and manageable. Inside each folder, I placed the following key files: `vttysh.conf`, `frr.conf` (the main configuration file for FRR) and `daemons`.

Once the files are set up, to interact with a particular router using `vttysh`, the following command needs to be executed on the router's terminal: `vttysh -N <router>` where `<router>` is the name of the router that I want to configure. This opens the `vttysh` shell for that particular router.

RIP.py

```
for router in [r1, r2, r3]:
    router.cmd("sysctl -w net.ipv4.ip_forward=1")

r1.cmd("/usr/lib/frr/frrinit.sh start 'r1'")
r2.cmd("/usr/lib/frr/frrinit.sh start 'r2'")
r3.cmd("/usr/lib/frr/frrinit.sh start 'r3'")
```

Mininet - Kathara

Both Mininet and Kathara use the same *frr.conf* files:

r1 - frr.conf

```
router rip
network 192.168.1.0/24
network 10.0.0.0/30
network 10.0.0.8/30
```

r2 - frr.conf

```
router rip
network 10.0.0.0/30
network 10.0.0.4/30
```

r3 - frr.conf

```
router rip
network 10.0.0.4/30
network 10.0.0.8/30
network 192.168.0.0/24
```

Kathara

It is necessary to start the frr service for each router, that is done with the following command that needs to be written inside the router *.startup* file. The same must be done in all the scenarios that follow

```
systemctl start frr
```

4.2.6 OSPF

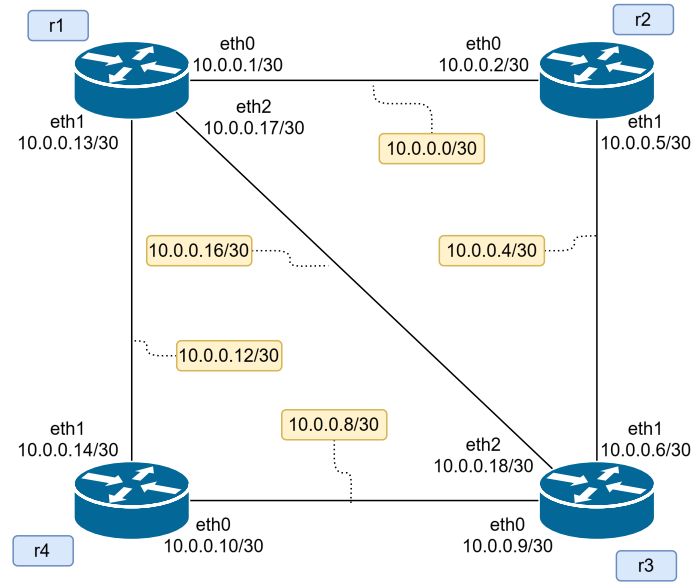


Figure 4.6: OSPF Routing Scenario

Mininet

Starting with this scenario, the connections between hosts are explicitly defined to prevent issues with incorrect ports or IP addresses. Below is an excerpt from *OSPF.py*:

OSPF.py

```
net.addLink(
    r1,
    r2,
    intfName1="r1-eth0",
    params1={"ip": "10.0.0.1/30"},
    intfName2="r2-eth0",
    params2={"ip": "10.0.0.2/30"},
)

for router in [r1, r2, r3, r4]:
    router.cmd("sysctl -w net.ipv4.ip_forward=1")

r1.cmd("/usr/lib/frr/frrinit.sh start 'r1'")
r2.cmd("/usr/lib/frr/frrinit.sh start 'r2'")
r3.cmd("/usr/lib/frr/frrinit.sh start 'r3'")
r4.cmd("/usr/lib/frr/frrinit.sh start 'r4'")
```

In order to make vtysh and ospfd work with Mininet I had to edit the file with the following path `/etc/frr/daemons` editing `ospfd=no` to `ospfd=yes`

Mininet - Kathara

Both Mininet and Kathara use the same *frr.conf* files:

r1 - frr.conf

```
router ospf
network 10.0.0.0/30 area 0.0.0.0
network 10.0.0.12/30 area 0.0.0.0
network 10.0.0.16/30 area 0.0.0.0
```

r2 - frr.conf

```
router ospf
network 10.0.0.0/30 area 0.0.0.0
network 10.0.0.4/30 area 0.0.0.0
```

r3 - frr.conf

```
router ospf
network 10.0.0.4/30 area 0.0.0.0
network 10.0.0.8/30 area 0.0.0.0
network 10.0.0.16/30 area 0.0.0.0
```

r4 - frr.conf

```
router ospf
network 10.0.0.8/30 area 0.0.0.0
network 10.0.0.12/30 area 0.0.0.0
```

4.2.7 BGP

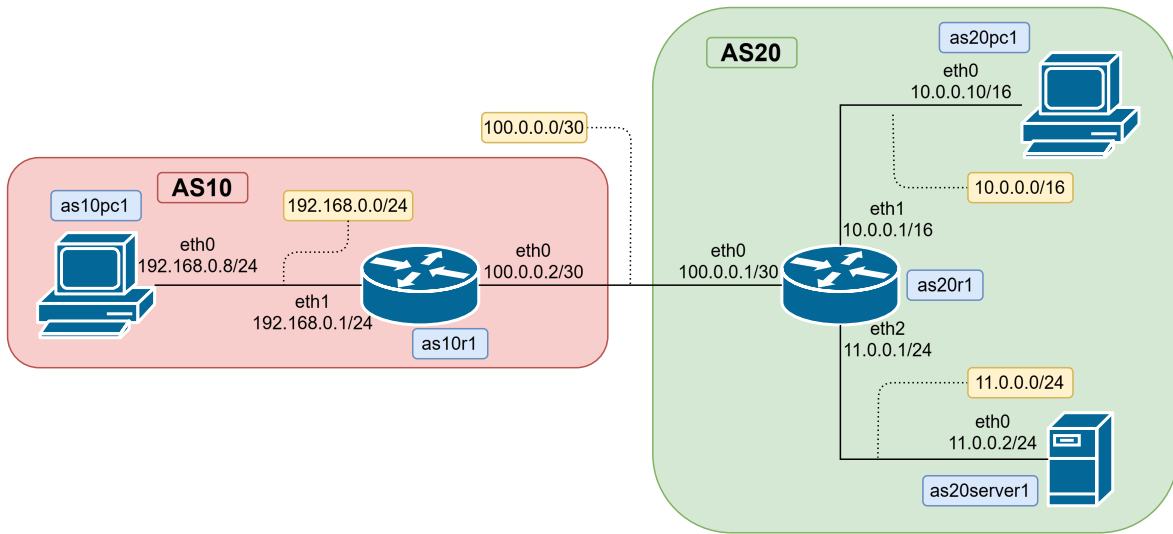


Figure 4.7: BGP Scenario with two Autonomous Systems

This scenario configures two Autonomous Systems (ASes) to interact using the Border Gateway Protocol (BGP). The objective is to assess how Mininet and Kathara manage inter-domain routing, focusing on the exchange of routing information and routing policy management between the ASes.

Mininet

For each router, the daemons file had to be modified to allow BGP to be propagated on the router of the neighbour AS

as10r1 - daemons

```
zebra=yes
bgpd=yes

vtysh_enable=yes
zebra_options=" -s 90000000 --daemon -A 127.0.0.1"
bgpd_options="  --daemon -A 100.0.0.2"
```

as20r1 - daemons

```
zebra=yes
bgpd=yes

vtysh_enable=yes
zebra_options=" -s 90000000 --daemon -A 127.0.0.1"
bgpd_options="  --daemon -A 100.0.0.1"
```

A further problem arose from this scenario, port 179 had to be enabled for BGP to function properly.

BGP.py

```

as10r1.cmd("iptables -A INPUT -p tcp --dport 179 -j ACCEPT")
as10r1.cmd("iptables -A OUTPUT -p tcp --sport 179 -j ACCEPT")

as20r1.cmd("iptables -A INPUT -p tcp --dport 179 -j ACCEPT")
as20r1.cmd("iptables -A OUTPUT -p tcp --sport 179 -j ACCEPT")

```

Mininet - Kathara

Both Mininet and Kathara use the same *frr.conf* files:

as10r1 - frr.conf

```

debug bgp keepalives
debug bgp updates in
debug bgp updates out
!
router bgp 10
  bgp router-id 100.0.0.2
  no bgp ebgp-requires-policy
  neighbor 100.0.0.1 remote-as 20
  neighbor 100.0.0.1 description Router as20r1
  !
  network 192.168.0.0/24

```

as20r1 - frr.conf

```

debug bgp keepalives
debug bgp updates in
debug bgp updates out
!
router bgp 20
  bgp router-id 100.0.0.1
  no bgp network import-check
  neighbor 100.0.0.2 remote-as 10
  neighbor 100.0.0.2 description Router as10r1
  neighbor 100.0.0.2 prefix-list customerIn in
  neighbor 100.0.0.2 prefix-list ispOut out
  !
  network 10.0.0.0/16
  network 11.0.0.0/24
  !
ip prefix-list customerIn permit 192.168.0.0/24
ip prefix-list ispOut permit 10.0.0.0/16
ip prefix-list ispOut permit 11.0.0.0/24

```

4.2.8 Multiple Solutions: BGP, OSPF, RIP and WebServer

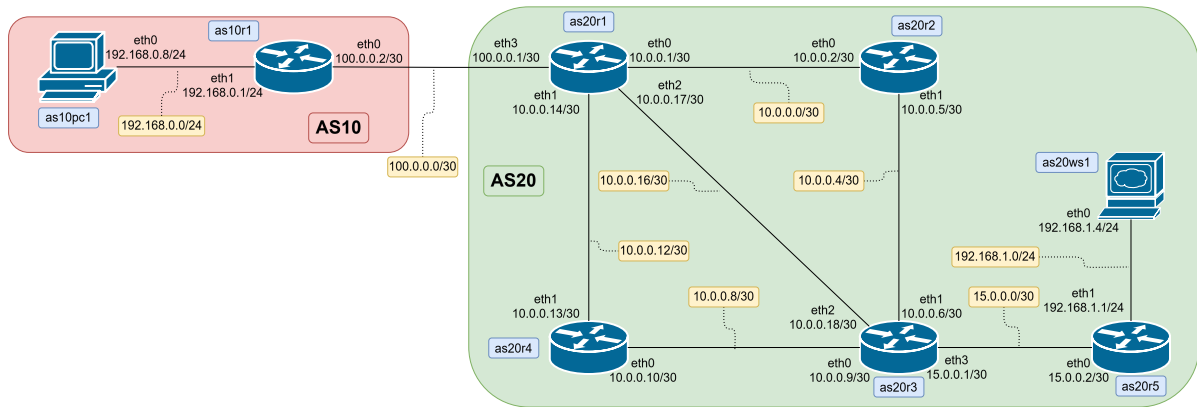


Figure 4.8: Complex Scenario combining BGP, OSPF, RIP Protocols and a Web Server

This scenario aims to demonstrate how both emulators perform in complex setups that integrate multiple protocols and feature a more intricate network topology. BGP operates between *as10r1* and *as20r1*, OSPF operates in the mesh of *as20r1*, *as20r2*, *as20r3*, and *as20r4*, while RIP enables connectivity between *as20r3* and *as20r5*, linking to the network hosting the web server.

Mininet

MultipleSolutions.py

```
as10pc1.cmd("ip route add default via 192.168.0.1")
as20ws1.cmd("ip route add default via 192.168.1.1")

for router in [as10r1, as20r1, as20r2, as20r3, as20r4, as20r5]:
    router.cmd("/usr/lib/frr/frrinit.sh start " + router.name)
    router.cmd("sysctl -w net.ipv4.ip_forward=1")
    router.cmd("iptables -A INPUT -p tcp --dport 179 -j ACCEPT")

as20ws1.cmd("cd /var/www/html && python3 -m http.server 80 &")
```

The above code consists of three main sections:

1. In the first section the default routes are created from the two PCs and the Web-Server
2. In the second section the *frr* service is started, forwarding enabled and port 179 configured
3. In the third and final section the server is started up on the host *as20ws1*

Mininet - Kathara

as10r1 - frr.conf

```
debug bgp keepalives
debug bgp updates in
debug bgp updates out

router bgp 10
no bgp ebgp-requires-policy
bgp router-id 100.0.0.2
neighbor 100.0.0.1 remote-as 20
neighbor 100.0.0.1 description Router as20r1
network 192.168.0.0/24
redistribute bgp
redistribute connected
redistribute rip
redistribute ospf
```

as20r1 - frr.conf

```
debug bgp keepalives
debug bgp updates in
debug bgp updates out

router ospf
network 10.0.0.0/30 area 0.0.0.0
network 10.0.0.12/30 area 0.0.0.0
network 10.0.0.16/30 area 0.0.0.0
redistribute bgp
redistribute connected
redistribute rip
redistribute ospf

router bgp 20
no bgp network import-check
bgp router-id 100.0.0.1
neighbor 100.0.0.2 remote-as 10
neighbor 100.0.0.2 description Router as10r1
neighbor 100.0.0.2 default-originate
neighbor 100.0.0.2 prefix-list customerIn in
neighbor 100.0.0.2 prefix-list ispOut out

network 0.0.0.0/0
network 10.0.0.0/30
network 10.0.0.16/30
network 10.0.0.12/30
redistribute bgp
redistribute connected
redistribute rip
redistribute ospf

ip prefix-list customerIn permit 192.168.0.0/24
ip prefix-list ispOut permit 0.0.0.0/0
```

as20r2 - frr.conf

```
router ospf
network 10.0.0.0/30 area 0.0.0.0
network 10.0.0.4/30 area 0.0.0.0
```

as20r3 - frr.conf

```
router ospf
network 10.0.0.4/30 area 0.0.0.0
network 10.0.0.8/30 area 0.0.0.0
network 10.0.0.16/30 area 0.0.0.0
redistribute bgp
redistribute connected
redistribute rip
redistribute ospf

router rip
network 15.0.0.0/30
redistribute bgp
redistribute connected
redistribute rip
redistribute ospf
```

as20r4 - frr.conf

```
router ospf
network 10.0.0.8/30 area 0.0.0.0
network 10.0.0.12/30 area 0.0.0.0
redistribute bgp
redistribute connected
redistribute rip
redistribute ospf
```

as20r5 - frr.conf

```
router rip
network 192.168.1.0/24
network 15.0.0.0/30
redistribute bgp
redistribute connected
redistribute rip
redistribute ospf
```

As seen in the contents of the files listed above, to ensure that the various routes propagate correctly between the routers, it was necessary to explicitly use the **redistribute** command

4.2.9 DNS

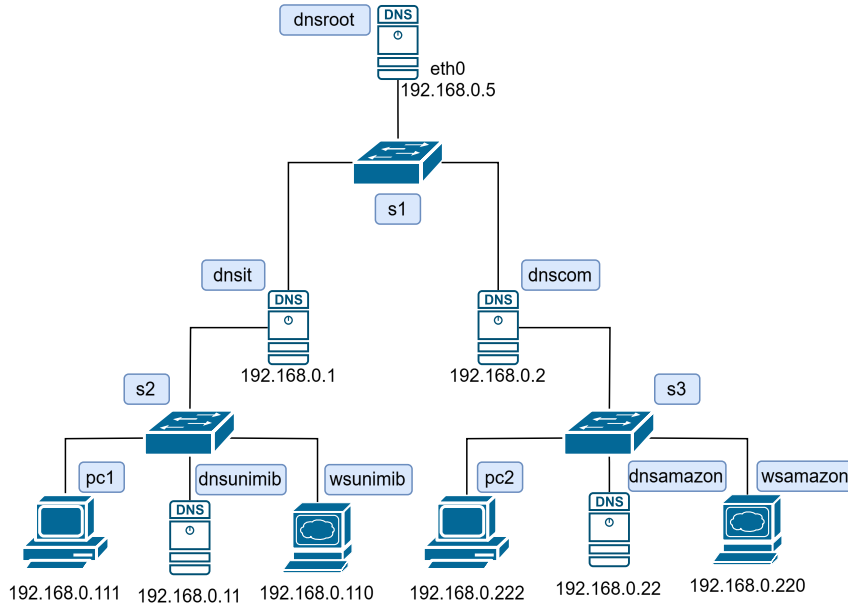


Figure 4.9: DNS Scenario with multiple Name Servers

This diagram illustrates the hierarchical structure of the Domain Name System (DNS) and its role in resolving domain names into IP addresses. At the top, the root DNS server directs queries down the hierarchy to authoritative and recursive DNS servers. These servers interact with the end-user devices, demonstrating how DNS requests propagate through different levels to retrieve the corresponding IP addresses. This scenario highlights the distributed and scalable nature of DNS, which ensures efficient and reliable domain resolution across the internet.

Mininet

It is not possible, without containers, to create isolated environments for each name server. Several attempts have been made, as in the previous cases, to emulate an isolated environment for each host but it was impossible to replicate, in Mininet, a DNS structure of that complexity. It is important to note that if there had been only one name server, this problem would not have occurred because records would have come from the single name server.

In an ideal setup, each host in the network would be isolated and could have independent DNS configurations and query handling. But without containers, Mininet cannot provide any type of virtual machine or process isolation in order to completely emulate a multi-server DNS setup. Each DNS server should work independently, but Mininet's default network topology cannot offer the same kind of isolation that Kathara does.

Due to the lack of containerization in Mininet, even the use of network namespaces or virtual interfaces to try to simulate an isolated environment for each host was not enough to create the required independence between DNS servers. The inability to isolate name servers may lead to conflicts when replicating complex DNS setups where different servers manage separate domains or zones.

Containernet To assess whether the issues encountered were indeed related to the absence of containers, Containernet was installed and configured. During this process, I discovered that the installation instructions provided on the official website were obsolete for Ubuntu 24.04, requiring a search for updated documentation. The correct setup details were ultimately found on the GitHub project page, which provided the contemporary configuration steps required for successful implementation. After overcoming these initial setup challenges, testing revealed immediate problems with the tools installed on the provided Docker images. After evaluating several options, the most efficient solution was to independently create custom Docker images tailored to my specific requirements. This approach proved successful, as it then became straightforward to implement the same configurations previously utilized in the Kathara environment, thus allowing for direct comparison between the two containerization platforms while maintaining consistent testing parameters.

```
build-images.sh
```

```
cat > dnsroot/Dockerfile << 'EOF'
    FROM ubuntu:20.04

    RUN apt-get update && \
        DEBIAN_FRONTEND=noninteractive apt-get install -y bind9 bind9utils
        ↪ dnsutils iproute2 iputils-ping net-tools && \
        apt-get clean && \
        rm -rf /var/lib/apt/lists/*

    RUN mkdir -p /var/run/named && \
        chown -R bind:bind /var/run/named && \
        mkdir -p /var/cache/bind && \
        chown -R bind:bind /var/cache/bind

    COPY named.conf.options /etc/bind/
    COPY named.conf /etc/bind/
    COPY db.root /etc/bind/

    RUN chown -R bind:bind /etc/bind

    EXPOSE 53/udp
    EXPOSE 53/tcp

    CMD ["/usr/sbin/named", "-g", "-c", "/etc/bind/named.conf", "-u",
        ↪ "bind"]
EOF
```

Kathara

The lab is composed of one folder and one file for each device in the network, according to the topology above. Kathara does not require the use of switches due to the usage of a single collision domain, which is why they are not considered in the current lab.

The *bind9* service is used to make DNS work properly, so every DNS server must contain the following line in its *.startup* file:

```
systemctl start bind9
```

Each DNS server folder contains a file named *named.conf.options*, which includes the following:

named.conf.options

```
options {
    directory "/var/cache/bind";
};
```

dnsroot folder:

db.root

```
$TTL 60000
@           IN  SOA      ROOT-SERVER.  root.ROOT-SERVER. (
                        2025020302 ; serial
                        28800      ; refresh
                        14400      ; retry
                        3600000    ; expire
                        0          ; negative cache ttl
                        )

@           IN  NS       ROOT-SERVER.
ROOT-SERVER. IN  A        192.168.0.5

it.         IN  NS       dnsit.it.
dnsit.it.   IN  A        192.168.0.1

com.        IN  NS       dnscom.com.
dnscom.com. IN  A        192.168.0.2
```

named.conf

```
include "/etc/bind/named.conf.options";

zone "." {
    type master;
    file "/etc/bind/db.root";
};
```

***dnsit* folder:**

db.it

```
$TTL 60000
@           IN  SOA      dnsit.it.  root.dnsit.it. (
                2025020302 ; serial
                28800      ; refresh
                14400      ; retry
                3600000    ; expire
                0          ; negative cache ttl
                )

@           IN  NS       dnsit.it.
dnsit.it.   IN  A        192.168.0.1

unimib.it.  IN  NS       dnsunimib.unimib.it.
dnsunimib.unimib.it. IN  A  192.168.0.11
```

db.root

```
.           IN  NS       ROOT-SERVER.
ROOT-SERVER. IN  A        192.168.0.5
```

named.conf

```
include "/etc/bind/named.conf.options";

zone "." {
    type hint;
    file "/etc/bind/db.root";
};

zone "it" {
    type master;
    file "/etc/bind/db.it";
};
```

dnscom folder:

db.com

```
$TTL 60000
@           IN  SOA     dnscom.com.  root.dnscom.com. (
                        2025020302 ; serial
                        28800      ; refresh
                        14400      ; retry
                        3600000    ; expire
                        0          ; negative cache ttl
                        )

@           IN  NS      dnscom.com.
dnscom.com. IN  A        192.168.0.2

amazon.com. IN  NS      dnsamazon.amazon.com.
dnsamazon.amazon.com. IN A 192.168.0.22
```

db.root

```
.           IN  NS      ROOT-SERVER.
ROOT-SERVER. IN A       192.168.0.5
```

named.conf

```
include "/etc/bind/named.conf.options";

zone "." {
    type hint;
    file "/etc/bind/db.root";
};

zone "com" {
    type master;
    file "/etc/bind/db.com";
};
```

dnsunimib folder:**db.it.unimib**

```
$TTL 60000
@           IN  SOA      dnsunimib.unimib.it.  root.dnsunimib.unimib.it. (
                                2025020302 ; serial
                                28          ; refresh
                                14          ; retry
                                3600000    ; expire
                                0           ; negative cache ttl
                                )

@           IN  NS       dnsunimib.unimib.it.
dnsunimib.unimib.it. IN  A   192.168.0.11

pc1.unimib.it. IN  A      192.168.0.111
wsunimib.unimib.it. IN  A  192.168.0.110
```

db.root

```
.           IN  NS       ROOT-SERVER.
ROOT-SERVER. IN  A       192.168.0.5
```

named.conf

```
include "/etc/bind/named.conf.options";

zone "." {
    type hint;
    file "/etc/bind/db.root";
};

zone "unimib.it" {
    type master;
    file "/etc/bind/db.it.unimib";
};
```

dnsamazon folder:

db.com.amazon

```
$TTL 60000
@           IN  SOA      dnsamazon.amazon.com.  root.dnsamazon.amazon.com. (
                                2025020302 ; serial
                                28          ; refresh
                                14400       ; retry
                                3600000    ; expire
                                15          ; negative cache ttl
                                )

@           IN  NS       dnsamazon.amazon.com.
dnsamazon.amazon.com. IN  A   192.168.0.22

pc2.amazon.com. IN  A      192.168.0.222
wsamazon.amazon.com. IN  A   192.168.0.220
```

db.root

```
.           IN  NS       ROOT-SERVER.
ROOT-SERVER. IN  A        192.168.0.5
```

named.conf

```
include "/etc/bind/named.conf.options";

zone "." {
    type hint;
    file "/etc/bind/db.root";
};

zone "amazon.com" {
    type master;
    file "/etc/bind/db.com.amazon";
};
```

***wsunimib* and *wsamazon* folders:** Both *wsunimib* and *wsamazon* folders contain the following files:

db.root

.	IN	NS	ROOT-SERVER.
ROOT-SERVER.	IN	A	192.168.0.5

named.conf

```
include "/etc/bind/named.conf.options";

zone "." {
    type hint;
    file "/etc/bind/db.root";
};
```

named.conf.options

```
options {
    directory "/var/cache/bind";
    allow-recursion { 192.168.0.0/24; };
    auto-dnssec off;
    dnssec-validation no;
    dnssec-enable no;
    dnssec-lookaside no;
    filter-aaaa-on-v4 yes;
    send-cookie no;
};
```


Chapter 5

Non-Functional Analysis Results

5.1 Performance Test Results

For each of the scenarios described in the previous chapter, some graphs will be presented showing the consumption of resources such as RAM and CPU, when performing simulations both in Mininet and Kathara. The proposed graphs will clearly highlight how each emulator uses system resources under different conditions.

These tests were done by executing two different Bash scripts: one for Mininet, one for Kathara, in order to automatically track the consumption of system resources for each scenario executed in a structured and reproducible way.

Mininet Data Collection For Mininet, the data of resource usage is extracted using the `ps aux` command. It lists all currently running processes along with the percentage usage of CPU and memory by each process. To accurately filter Mininet-related processes, several keywords are used within the monitoring script.

The monitoring script filters for processes containing keywords such as `python` (as Mininet runs using Python scripts), `frr` (FRRouting, an open-source routing stack common in network emulation), `mininet` (main Mininet processes), `ovs` (Open vSwitch that manages virtual switches), `controller` (SDN controllers often running with Mininet), `mn` (Mininet processes shorthand), and `openv` (variant for Open vSwitch process names).

The script explicitly excludes processes containing the keyword `code` (Visual Studio Code) to prevent background applications from distorting CPU and memory usage metrics, ensuring cleaner data collection results.

Kathara Data Collection Kathara's data collection mirrors Mininet's approach but adapts filtering criteria to its environment. The script isolates processes based on the following keywords:

- *kathara* – It is the main process running and managing network emulation.
- *docker* – Kathara uses containerized environments, Docker processes are highly relevant in the management of individual virtual host executions.

For both Mininet and Kathara, overall resource usage is assessed by aggregating the resource consumption of all active processes (Mininet) or all running containers (Kathara). This ensures a comprehensive view of how each emulator affects system performance during execution.

5.1.1 Scenarios Structure

Mininet Monitoring Resource monitoring for Mininet is started a few seconds before the actual start of the scenario. This is achieved by running a dedicated logging script via *python3*, which starts CPU and memory usage tracking just before the initialization of Mininet. Once the network scenario has finished, Mininet automatically terminates when the execution of the `net.stop()` command is reached in the Python script. However, the logging script is kept running for some seconds after the termination of the scenario. This ensures that all transient resource spikes, such as those that may occur at the initialization of the network, and steady-state resource usage are recorded.

Kathara Monitoring Kathara uses a similar setup for the monitoring process, but it requires another set of commands to initialize and shut down the network scenario. The process begins with `kathara lstart`, which establishes the virtual network and launches the associated containers. After deployment and execution of the network, continuous monitoring of resource consumption is performed until the end of the simulation. `kathara wipe` is run to properly terminate the scenario, releasing all allocated resources and removing the containers created during execution. Just like in the Mininet setup, the logging script keeps running for a few seconds after the termination command is given to capture any remaining system activity before stopping.

Another important point is that, in the case of Kathara, an alternative approach would have been relying on docker stats, a Docker tool that provides real-time metrics related to CPU, memory, and network usage for running containers. However, to maintain coherence between the methodologies of data collection for Mininet and Kathara, the same approach based on `ps aux` has been chosen for both emulators. This will make sure that the data is actually collected on the same criteria and can allow for proper and realistic comparison between these two network-emulation platforms.

5.1.2 Hardware Specifications

To ensure transparency in performance metrics, the hardware configuration is detailed below:

- Device Model: Lenovo IdeaPad Gaming 3 15ACH6
- Processor: AMD Ryzen™ 7 5800H with Radeon™ Graphics × 16
- Graphics Card: NVIDIA GeForce RTX™ 3060 Laptop GPU
- Operating System: Ubuntu 24.04.2 LTS

These specifications directly affect the CPU and memory usage measurements presented in this analysis.

5.1.3 Simple Network with Two Hosts

Open all terminals and simultaneously execute `ping 'ip' -c 10` from both hosts to test connectivity with 10 ICMP packets.

Mininet

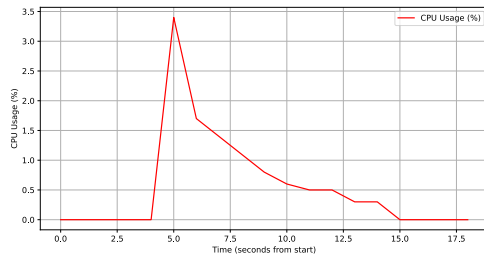


Figure 5.1: Two Host Mininet CPU Usage

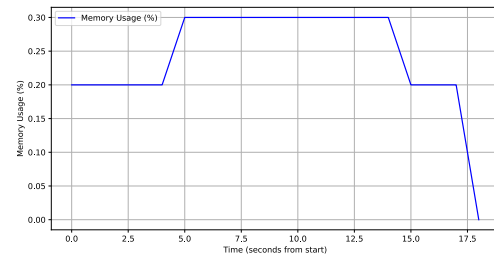


Figure 5.2: Two Host Mininet Memory Usage

Kathara

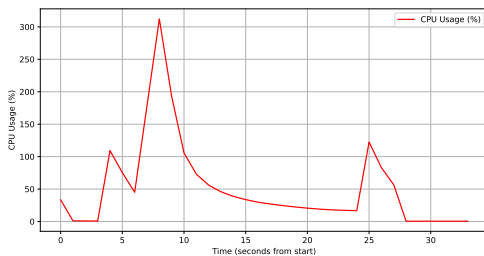


Figure 5.3: Two Host Kathara CPU Usage

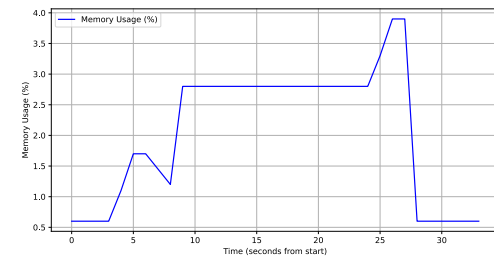


Figure 5.4: Two Host Kathara Memory Usage

5.1.4 Static Routing

Open all terminals and simultaneously execute `ping 'ip' -c 10` from both hosts to test connectivity with 10 ICMP packets.

Mininet

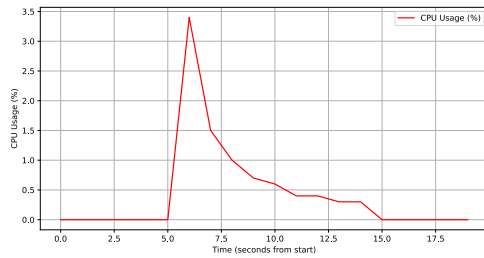


Figure 5.5: Static Routing Mininet CPU Usage

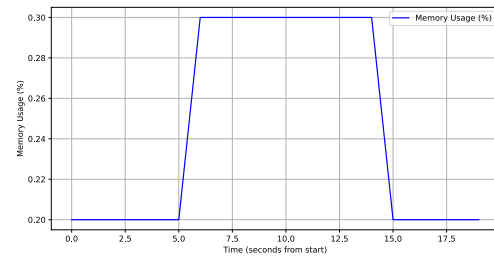


Figure 5.6: Static Routing Mininet Memory Usage

Kathara

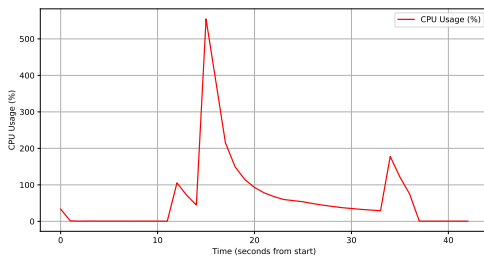


Figure 5.7: Static Routing Kathara CPU Usage

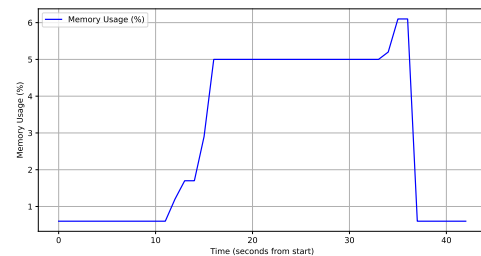


Figure 5.8: Static Routing Kathara Memory Usage

5.1.5 Client Server

Open all terminals, start iperf3 server in daemon mode on ws1 using `iperf3 -s -D`, then run a 30-second throughput test from pc1 with `iperf3 -c 10.0.5.2 -t 30`.

Mininet

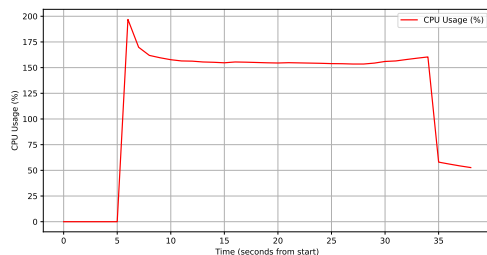


Figure 5.9: Client-Server Mininet CPU Usage

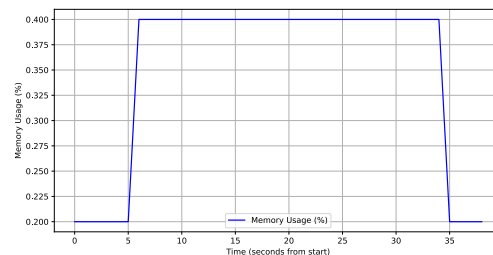


Figure 5.10: Client-Server Mininet Memory Usage

Kathara

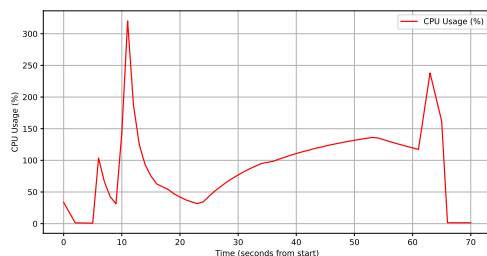


Figure 5.11: Client-Server Kathara CPU Usage

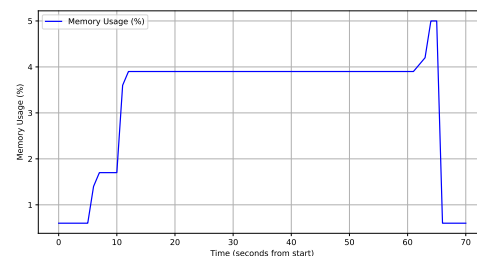


Figure 5.12: Client-Server Kathara Memory Usage

5.1.6 Load Balancing

Open all terminals, launch iperf3 server daemons on the load balancer and all web servers with `iperf3 -s -D`, then simultaneously conduct 30-second bandwidth tests from pc1 and pc2 targeting 10.0.0.4 using `iperf3 -c 10.0.0.4 -t 30`.

Mininet

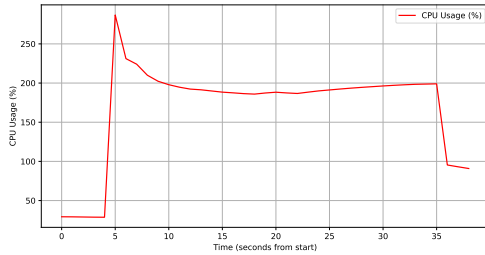


Figure 5.13: Load Balancing Mininet CPU Usage

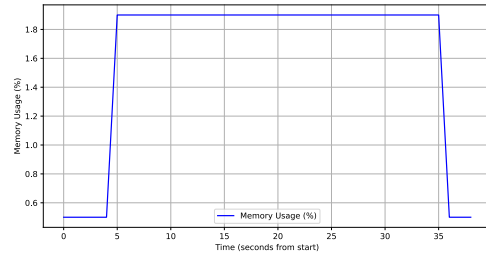


Figure 5.14: Load Balancing Mininet Memory Usage

Kathara

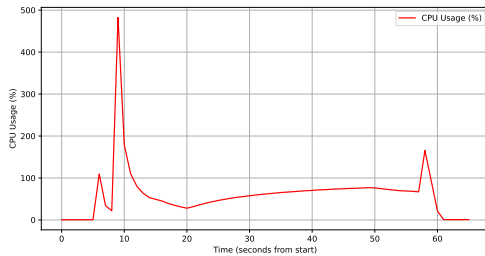


Figure 5.15: Load Balancing Kathara CPU Usage

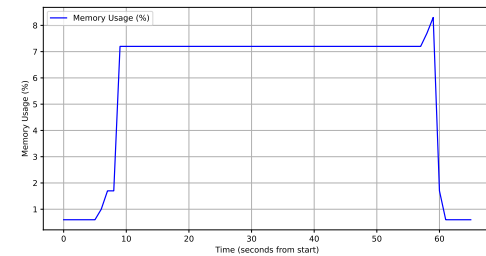


Figure 5.16: Load Balancing Kathara Memory Usage

5.1.7 RIP

Open all terminals and run `ping 192.168.x.x -c 10` simultaneously from pc1 and pc2, sending 10 ICMP packets to test connectivity to the specified host.

Mininet

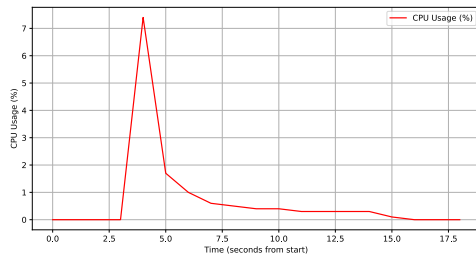


Figure 5.17: RIP Mininet CPU Usage

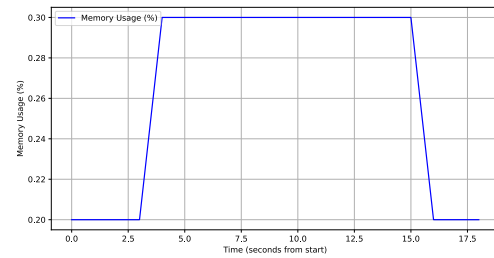


Figure 5.18: RIP Mininet Memory Usage

Kathara

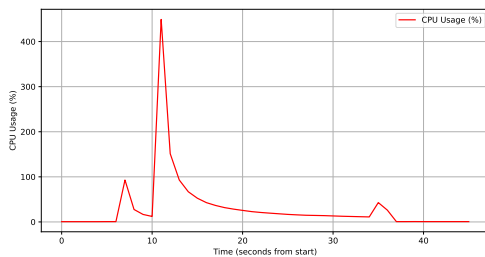


Figure 5.19: RIP Kathara CPU Usage

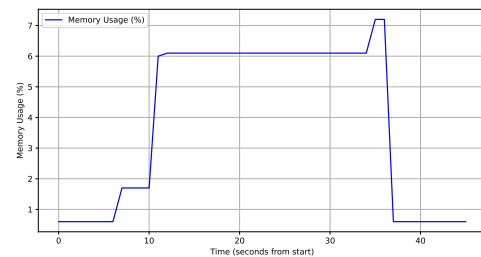


Figure 5.20: RIP Kathara Memory Usage

5.1.8 OSPF

Open all terminals, allow 60 seconds for OSPF route propagation, then simultaneously execute `ping -I eth0 10.0.0.6 -c 30` from r1 and `ping -I eth0 10.0.0.14 -c 30` from r2, using the eth0 interface as the source.

Mininet

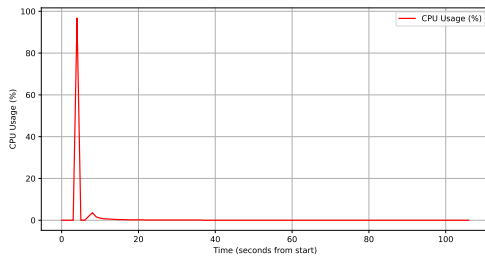


Figure 5.21: OSPF Mininet CPU Usage

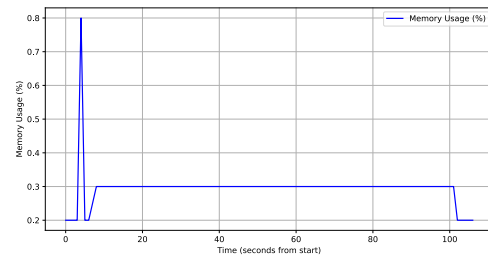


Figure 5.22: OSPF Mininet Memory Usage

Kathara

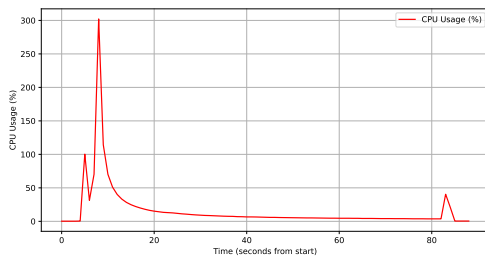


Figure 5.23: OSPF Kathara CPU Usage

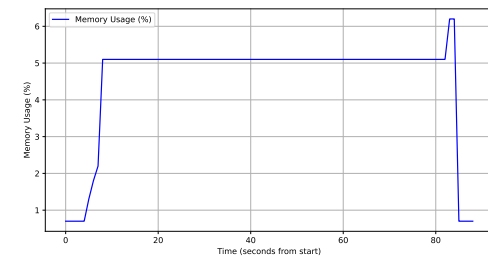


Figure 5.24: OSPF Kathara Memory Usage

5.1.9 BGP

Open all terminals and execute `ping 11.0.0.2 -c 20` from `as10pc1`, sending 20 ICMP packets to test connectivity.

Mininet

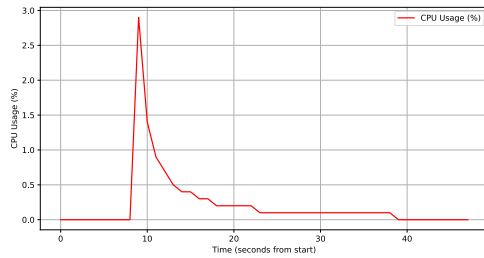


Figure 5.25: BGP Mininet CPU Usage

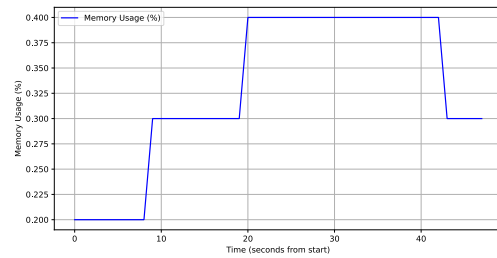


Figure 5.26: BGP Mininet Memory Usage

Kathara

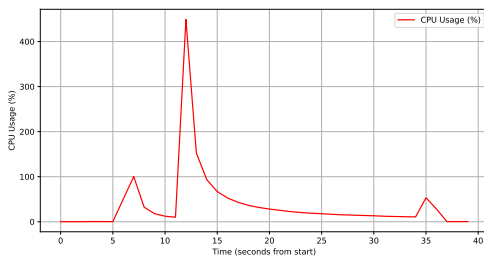


Figure 5.27: BGP Kathara CPU Usage

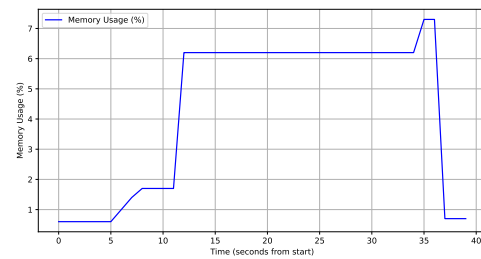


Figure 5.28: BGP Kathara Memory Usage

5.1.10 Multiple Solutions: BGP, OSPF, RIP and WebServer

Open all terminals, start an iperf3 server in daemon mode on as20ws1 using `iperf3 -s -D`, wait 30 seconds for setup completion, then run a 30-second throughput test from as10pc1 with `iperf3 -c 192.168.1.4 -t 30`.

Mininet

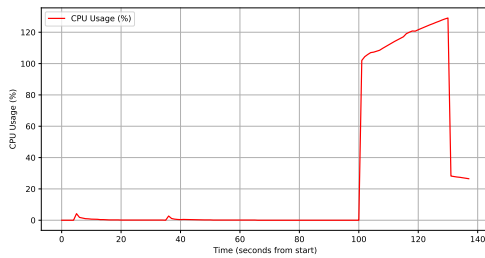


Figure 5.29: Multiple Solutions Mininet CPU Usage

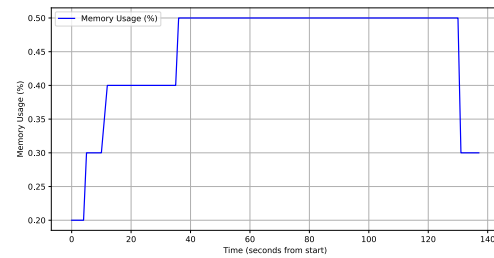


Figure 5.30: Multiple Solutions Mininet Memory Usage

Kathara

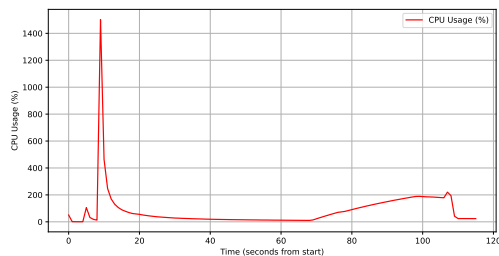


Figure 5.31: Multiple Solutions Kathara CPU Usage

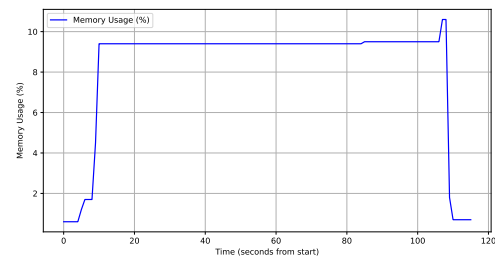


Figure 5.32: Multiple Solutions Kathara Memory Usage

5.1.11 DNS

Kathara

Open all terminals, start BIND9 DNS service on the DNS servers, wait 10 seconds for initialization, then run `ping -n wsamazon.amazon.com -c 20` on pc1 and `ping -n wsunimib.unimib.it -c 20` on pc2.

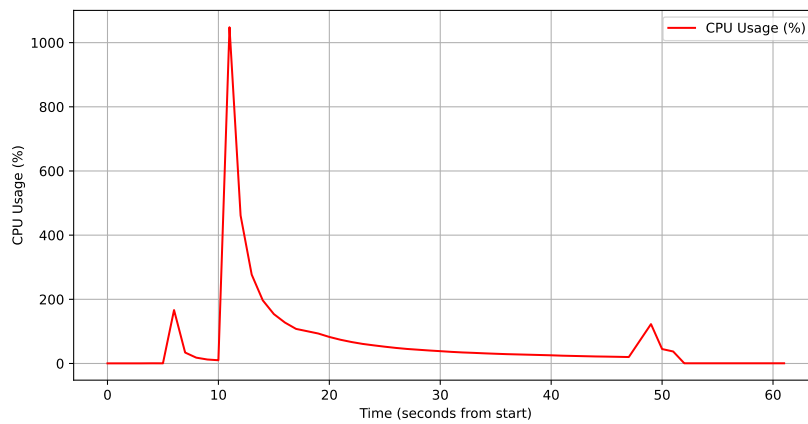


Figure 5.33: DNS Kathara CPU Usage

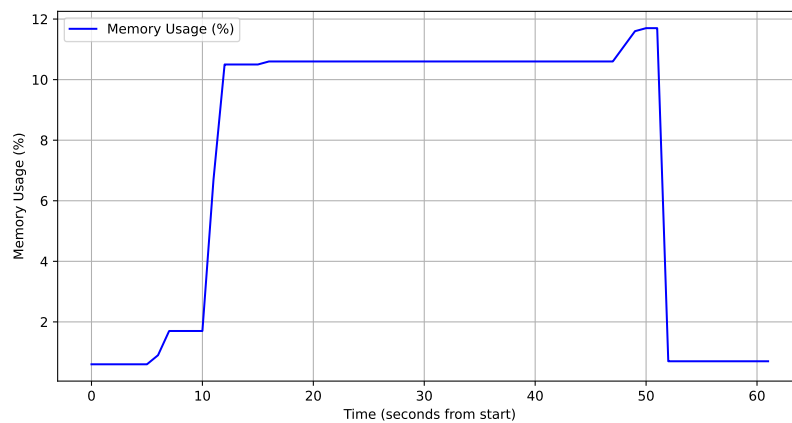


Figure 5.34: DNS Kathara Memory Usage

5.2 Comparative Analysis Based on Benchmarking Data

It is important to clarify that the percentage of CPU consumption is calculated based on the utilization of each core individually. In several cases, the recorded consumption exceeds 100%, which indicates that multiple cores are being utilized simultaneously. This is a common occurrence in multi-core processors when tasks are distributed across different processing units.

Another key observation is over the memory usage. The baseline memory consumption is never zero. This is evident in Mininet, where the measurement of memory consumption includes some services that are running before the scenario started. That is because, Mininet does not run in complete isolation but it runs in an environment where some essential services are already running and keep running in the background. On the other side, Kathara behaves differently: initially, only the minimal base service is active. Most of the additional services and processes are launched only after the scenario is initiated, which leads to a more incremental memory consumption pattern.

From the above graphs, it is easy to understand that Mininet requires significantly fewer resources compared to Kathara. This is by default because Mininet depends on using system components that are already up and running instead of initializing new instances for each network element.

On the other end Kathara follows a different approach. While creating a dedicated, isolated environment for each host (container) results in increased resource consumption, this provides solid isolation among network elements but at higher CPU and memory costs, which increases when dealing with complex scenarios or running several virtual nodes.

Performance Analysis - Mininet Mininet's performance exhibits a sharp increase in resource consumption within the first seconds of the scenario: this is while the virtual network is being brought up and configured. After a scenario is started and running, CPU consumption will gradually decrease while memory consumption will be stable during all the execution.

One particularly interesting case happens when running the `iperf3` command (a popular network performance measurement tool). CPU usage during the execution of `iperf3` remains high since most of the computations for handling network traffic generation and throughput measurements take place. From this trend, memory usage stays at the same level during most phases of the execution, while CPU consumption always shows a peak during startup, with a progressive decrease apart from those points in time where network traffic is generated.

Performance Analysis - Kathara When analysing Kathara's resource utilization, an interesting pattern related to CPU usage can be outlined. In contrast with Mininet, where the peak is observed mainly at the beginning, Kathara shows two clear CPU spikes during its initialization: The first, smaller spike occurs with the `kathara lstart` is called as Kathara prepares all the infrastructure that will be needed to run the virtual network environment but does not initialize the hosts yet.

The second, more significant peak happens during the actual startup phase, when every containerized host is instantiated and configured.

Later in the execution, a third peak can be observed toward the end of the scenario. This corresponds to the execution of `kathara wipe`, a command that releases all allocated

resources and removes the instantiated containers.

A similar pattern is seen in memory utilization. The memory consumption increases at the beginning of the scenario, while the network environment is set up. A second increase happens when the individual hosts are started. After the setup phase, memory usage settles down and remains the same during execution. Finally, during the *wipe* phase, the memory usage rises for a moment before it goes back to its original value. This brief spike right before release might be due to the overhead introduced by container removal and resource deallocation.

Conclusion Based on the above comparison, it seems there exist fundamental differences in system resource consumption by Mininet and Kathara. Mininet’s lightweight architecture enables efficient CPU and memory use, relying on pre-installed system components and avoiding the overhead introduced by full isolation. In contrast, Kathara prioritizes stronger isolation through containerized execution at the cost of higher resource demands.

Chapter 6

Functional Analysis

6.1 Architectural Comparison

Mininet and Kathara both have extremely different approaches to network emulation, each with its own architectural strengths and trade-offs.

Mininet uses lightweight Linux namespaces and a shared kernel to create virtual hosts, an efficient yet less isolated method.

Kathara, on the other hand, utilizes Docker containers, which provide full isolation for each node at the cost of additional overhead.

These variations affect how network topologies are formed, services are set up, and how scalable each piece of software is for a particular situation. The following table shows these architectural differences.

Aspect	Mininet	Kathara
Emulation Method	Uses Linux namespaces to create virtual hosts and links	Exploits Docker containers to isolate network nodes
Virtualization Approach	Lightweight emulation with shared kernel	Full containerization for each node
Network Topology Definition	Defined programmatically using Python API	Defined through configuration files and directories
Process Isolation	All nodes share the same underlying file system	Each node has an independent file system and environment
Scalability	Limited by system resources since there is no isolation	More scalable due to container-based isolation
Customizability	Requires manual installation of network services	Supports pre-configured container images with services
Underlying Network Stack	Uses Open vSwitch (OVS) to simulate SDN environments	Uses Linux network namespaces within Docker to create virtual networks

Table 6.1: Architectural Differences Between Mininet and Kathara

6.2 Flexibility and Extensibility

A critical factor in examining network emulation frameworks is their extensibility and flexibility. Their ability to integrate with additional components, expand functionality, and adapt to various platforms significantly enhances their utility.

This section examines Mininet’s and Kathara’s extensibility and flexibility according to the extensions and contributions of their respective communities. Comparison reveals how each system can be extended and customized to meet special networking requirements with regard to being portable on other platforms.

6.2.1 Mininet

Mininet has established itself as a key tool in network research. Over time, the open-source community has developed a lot of extensions to improve its functionality and make network configurations easier. While this study focuses on a direct comparison between Mininet and Kathara without integrating additional tools, it is worth mentioning some of the available extensions.

The GitHub repository for Mininet extensions [32] lists some packages designed to simplify Mininet installation on different operating systems. However, many of these links are no longer active, which indicates a lack of maintenance. One exception is a porting of Mininet to *TinyCore Linux*.

Several tools have been developed to make Mininet more user-friendly. One of those tools is *Visual Description Network*, a graphical interface designed to automate the generation of Mininet and OpenFlow scripts although its official download link is no longer active too. Other extensions are *Chef cookbook* for automating Mininet installation and *Mininet Editor*, it offers a web-based interface for building Mininet topologies, featuring a modern GUI, real-time error checking, address plan exporting, and script importing.

Mininet also features several specialized variants that extend its core functionalities. A prime example is *IPMininet*, which is designed to emulate complex IPv4 and IPv6 networks. This extension introduces new network classes, such as Routers, and automates many network configuration aspects, including IP addressing and router configurations, reducing manual setup effort.

Another important Mininet extension is *Mininet-WiFi* [33], which allows users to emulate wireless networks. This extension introduces wireless stations, access points, and mobility features, making it an excellent tool for testing software-defined wireless networks.

Another extension is *Maxinet* [34], which enhances Mininet by enabling distributed emulation across multiple physical machines. This feature allows for larger and more complex network topologies to be tested, making it valuable for large-scale experiments.

Containernet *Containernet* [35, 36] is one of the most famous Mininet extensions, even though it is not officially listed on the Mininet GitHub page. It is a fork of Mininet that adds support for Docker containers, allowing users to deploy containerized hosts within the emulated network. This feature makes it particularly useful for testing cloud-native applications and micro services in a controlled environment.

Containernet enables real application containers to be used as network nodes, offering a more realistic testing environment compared to traditional Mininet hosts. This functionality makes it a valuable alternative to Kathara, it can also be a solution to some problems encountered during the testing phase in previous chapters.

During development work with Mininet and recognition of its limitations, there was increasing interest in testing Containernet to evaluate its effectiveness in addressing these problems. Although the initial configuration was not straightforward due to discrepancies between the official documentation and GitHub resources (as mentioned in the implementation section of various scenarios), the overall setup process eventually proved to be quite simple.

To simplify host management, it was convenient to independently create Docker images for the hosts, installing all necessary plug-ins to develop a complete test suite.

Throughout testing, the hypothesis was confirmed: Containernet effectively resolves Mininet's limitations. The containers build in a reasonably short time (15-20 seconds for the DNS scenario), and the Python code requires only a few additional instructions for managing containerized hosts, while the rest remains similar to Mininet.

Another particularly useful feature is the ability to automate folder creation for each host and mount their respective filesystems, solving the issue of having to use the system filesystem to configure various services.

6.2.2 Kathara

Just like Mininet, Kathara has a variety of extensions and tools created by its developer community to improve its capabilities. One of the most significant extensions is *Megalos* [28, 29], a Kubernetes-based version of Kathara. Megalos introduces changes in system design and compatibility, allowing users to deploy network topologies dynamically and scale them efficiently.

Kathara also provides a collection of prebuilt Docker images [37]. These images include:

- **kathara/base**: The base image, containing essential network tools and services like BIND and Apache.
- **kathara/quagga**: Adds the Quagga routing suite for network protocol emulation.
- **kathara/frr**: Integrates the FRRouting (FRR) suite for advanced routing protocol support.
- **kathara/openbgpd**: Provides support for OpenBGPD, a BGP implementation.
- **kathara/krill**: Includes Krill, an RPKI Certificate Authority for secure routing.
- **kathara/routinator**: Adds Routinator for RPKI validation.
- **kathara/rpki-client**: Integrates OpenBGPD's RPKI Client for route validation.
- **kathara/bird**: Provides support for the BIRD routing daemon.
- **kathara/rift-python**: Implements the Routing In Fat Trees (RIFT) protocol for large-scale networks.
- **kathara/sdn**: Adds Open vSwitch (OVS) and the Ryu SDN controller, enabling software-defined networking.
- **kathara/p4**: Supports the Behavioural Model (bmv2) for P4 programmable switches.
- **kathara/pox**: Introduces POX, a Python-based SDN controller.

- **kathara/scion**: Implements SCION, a next-generation network architecture for scalability and security.

This extensive set of images highlights Kathara’s developers’ focus on compatibility with some of the most used network services and protocols.

Another useful extension is the *Kathara Network Plugin* [38], a Docker plug-in that facilitates the creation of pure Layer 2 LANs using either Virtual Distributed Ethernet (VDE) or traditional Linux bridges. This plug-in improves network flexibility and simplifies the setup of complex network scenarios.

Kathara has also some external tools [39] created by its developers or by the active community, including:

- *Netkit Lab Generator*, a tool, built by its developers, to automatically generate network configurations.
- *VFTGen*, which creates Fat Tree topologies for large-scale network simulations.
- *Tacatà*, a Python-based script that simplifies the creation of Netkit and Kathara labs.
- *net-vis*, a visualization tool that parses lab configuration files and generates network diagrams.
- *kathara-lab-starter*, a JSON-based tool that accelerates the setup of Kathara environments by minimizing configuration redundancies.

Both Mininet and Kathara have a lot of extensions and tools that enhance their flexibility and adaptability. Mininet, thanks to its long-standing presence in academic and industrial research, boasts a wide range of third-party contributions. On the other hand, Kathara provides a modern, container-based approach that integrates with various networking technologies. The choice between the two ultimately depends on the specific requirements of the network emulation scenario, such as container support, software-defined networking, or IPv6 functionality.

6.3 Ease of Use and Documentation

6.3.1 Mininet

Installation and Requirements Mininet offers multiple installation options: a pre-configured virtual machine from the official website, manual installation from source, or package manager installation. The virtual machine option eliminates compatibility issues and enables immediate use.

To verify installation, Mininet provides a simple command-line test utility:

```
sudo mn --switch ovsbr --test pingall
```

Mininet requires Python and is exclusively compatible with Unix-based operating systems [40], with Ubuntu being the recommended environment since most official documentation and tutorials focus on Ubuntu.

API and Usability Mininet’s high-level API simplifies host definition and network topology construction. While basic Python knowledge is necessary to leverage the API and execute network scripts, the interface is intuitive and allows straightforward configuration of hosts, switches, and links. This facilitates the creation of diverse network scenarios without requiring extensive networking command expertise.

Container Limitations and Workarounds A significant limitation of Mininet is its incompatibility with containerization. Instead, it relies on system-level services installed on the host system, which complicates deployment of network services requiring isolated file systems.

Two workarounds exist: the official approach documented in the *bind.py* example [13], and a more practical solution used in previous chapters using separate directories for each host to simulate isolated file systems. However, this non-containerized approach increases host management complexity, especially when implementing various network services, as dependencies and configurations must be manually managed across hosts sharing the same system.

Features and Compatibility Mininet includes built-in testing tools like *pingall* and *iperf* for easy connectivity verification and performance measurement. It integrates seamlessly with Open vSwitch (OVS) [41] and SDN controllers such as OpenDaylight [42], Ryu [43], and RipL-POX [44], making it valuable for SDN research and development. The platform’s extensibility through custom controllers and scripts enables users to design and test novel networking paradigms.

Community and Research Impact As a long-established network emulation tool, Mininet enjoys widespread adoption, particularly in SDN domains, supported by an active community. Resources include the *mininet-discuss* mailing list, an extensive FAQ section, and active presence on Stack Overflow and GitHub, where users can troubleshoot issues and access community-contributed scripts [45]. As of February 13, 2025, the keyword *Mininet* appears in 1,109 IEEE Xplore articles [46], demonstrating its significant research impact. This prominence has spawned numerous adaptations and extensions enhancing its functionality, including additional SDN controller support and integration with machine learning tools for network optimization.

6.3.2 Kathara

Platform Support and Docker Integration Kathara supports multiple platforms including Windows, macOS, and Linux [47]. Its key advantage is Docker integration, providing isolated environments for each network node, enabling scalable modeling of complex network scenarios. This containerized approach ensures each node operates in a sandboxed environment, preventing dependency conflicts and simplifying network service deployment.

Installation processes vary by platform. On Linux, differing distribution requirements typically involve adding Kathara to a package manager. Docker, a prerequisite due to Kathara’s foundation, must be pre-installed and configured. [48]

User Experience and Tools In Ubuntu, Kathara installation is straightforward, allowing network scenario execution without system-level modifications. Docker integration ensures environmental consistency across installations, minimizing compatibility issues. The KatharaFramework GitHub repository offers introductory tutorials for setup guidance, complemented by *Netkit Lab Generator*, a graphical tool simplifying scenario configuration. This tool allows developers to easily set up network services and protocols to generate usable networks in labs. The application also automates generation of required configuration files and individual file systems per host to simplify a process that would otherwise be tedious. The application is also capable of automatically generating a graphical view of the network topology to facilitate detection of potential configuration errors.

Setup Challenges and Containerization Benefits Initial Kathara lab setup can be challenging due to the configuration requirements of various file types, i.e., *.startup* and *.conf*, and per-host directory structures. However, the Lab Generator streamlines these processes, clarifying configuration file structures and reducing setup time.

Kathara’s containerization support, unlike Mininet’s, simplifies designing isolated environments for each node in the network. The functionality facilitates more realistic emulations of networks in that each node is given its unique file system, required installation packages, and services without interference from other nodes. Such a capability of Kathara is of great use in applications that involve routing protocols, distributed systems, and security audits.

Community, Documentation and Future Potential While Kathara has a smaller community than Mininet due to its relative newness, it offers comprehensive and often superior documentation. Its adoption by educational institutions [49] has generated numerous laboratory exercises guiding users through network scenario setup, with many scenarios developed by community members beyond the original development team.

Kathara undergoes continuous maintenance with new features and improvements based on user feedback. On GitHub, there is a *Discussions* section where developers and users interact for support and knowledge sharing aiding new users effectively. Accompanying documentation offers in-depth use cases, trouble shooting tips and example configuration files aiding new users effectively.

As of February 13, 2025, *Kathara* appears in only three different articles in the IEEE Xplore database. There are more citations in the official website of Kathara, though. This indicates that Kathara is still in its early stages, whereas Mininet has already estab-

lished itself as a well-known and extensively utilized tool in the field of network emulation. The constant improvement compared to Kathara, coupled with its increasing use, indicates that it has a high potential to be a serious rival to Mininet in the coming future. If more learning institutions and learning centres start using Kathara in their curricula and in their learning process, adoption would get a dramatic boost, resulting in a more active body of developers and users.

6.3.3 Summary

Feature	Mininet	Kathara
Platform Support	Unix-based systems (Linux)	Windows, macOS, Linux
Installation	Pre-configured VM, package manager, or manual installation	Requires Docker, different setup steps for each OS
Container Support	No, relies on system-wide services	Yes, uses Docker for isolated environments
Ease of Use	Requires Python knowledge, API for scripting	More complex manual setup, but Lab Generator simplifies it
Configuration Complexity	Services must be installed manually, no isolation between hosts	Uses dedicated file systems per host, simplifying service management
Graphical Interface	No official GUI	Netkit Lab Generator provides a graphical configuration tool
Community and Support	Large, active community, mailing list, Stack Overflow, GitHub	Smaller but growing community, GitHub discussions, university-backed documentation
Documentation	Well-documented, but some areas lack details	Very detailed, including university-developed labs
Academic Adoption	1,109 IEEE Xplore publications (as of Feb 2025)	3 IEEE Xplore publications, but additional references on the official site

Table 6.2: Comparison of Mininet and Kathara

Chapter 7

Conclusion and Future Work

7.1 Recommendations

Based on the experience gained while working with both emulators, as well as the findings reported in previous chapters, the following recommendations are provided from a completely objective perspective, relying on the collected data and acquired knowledge.

7.1.1 Mininet

Mininet has been a great option to set up easy network scenarios. It is a light and powerful tool that enables the creation of virtualized network topologies in a single machine, which is very effective for testing and experimenting with network configurations. However, integrating external services like FRRouting (FRR) complicates the setup phase.

While the installation of Mininet itself is simple, getting it to work with more sophisticated services can require some additional effort and precise configuration. This can be simplified with a virtual machine. A VM will ensure that the environment is properly configured, which can reduce the likelihood of having issues during the creation and execution of various scenarios. While it might be difficult to make Mininet work properly with external services like FRR, it is still well-liked since it is versatile and simple to use for uncomplicated network configurations.

One of the characteristics that distinguish Mininet is the fact that it offers a very useful API. This API makes it easy to create and manage network topologies, allowing users to easily install and modify network scenarios. This could end up saving a lot of time, especially when working with more complex projects. Mininet also uses fewer resources compared to some of the other tools, so it is an option for users who have less powerful hardware.

Mininet is recommended to anyone interested in simulating simple network setups or experience configuring network services. It is a very good option for anyone who needs a lightweight, customizable tool. For users who are more interested in experimentation or research with low resource usage, Mininet is also a very good choice.

However, for teaching or educational purposes, Mininet might not be the most suitable option. The lack of pre-built scenarios could mean that users would need to spend time understanding the environment before they can carry out experiments. This could be time-consuming and may bring inconvenience in distributing course materials to students because they would need to learn how to establish things themselves.

7.1.2 Kathara

Kathara takes a different approach to network emulation. At first, it might seem more complicated to use, but after understanding its features and learning how to work with the pre-built labs, it becomes much easier to manage. The key to using it effectively is understanding its configuration system and how the files are organized within each host's folder. Familiarity with this structure makes it quite straightforward.

Kathara is a perfect solution for those who wish to be exposed to network emulation without necessarily having to deal with low-level service configurations. It is a must for self-learners or anyone interested in trying out network configurations without so much focus on the technicalities. It is also the right choice for those with access to further computing resources since it allows the testing of more complex network topologies in a sandbox environment.

One of the things that makes Kathara so great from an educational perspective is how simple it is to use. There are loads of pre-written labs and tutorials that come with it, so there is no need to design a new lab starting from scratch. Netkit Lab Generator helps Kathara generate several scenarios that are easily installed and can be used for experimentation or teaching. This is very useful for teachers who must develop transparent, reproducible scenarios for students.

In terms of hardware requirements, Kathara is slightly more demanding. It can function well on computers with higher processing power, which is ideal for bigger simulations or stress-testing complex network setups. If working in an environment with access to these resources, Kathara is a suitable choice.

Kathara offers arguably the most straightforward model to share network simulations. Using its well-documented and pre-configured labs, it is easy to build and share education content that would be easily modified by students. Since the simulations are portable, students can easily download and run the same configurations, learning through experimentation of actual network configurations.

Overall, Kathara is an excellent choice for those looking for a simple and convenient method of learning how to utilize network emulation. Its helpful documentation and pre-configured labs make it perfect for independent study or educators looking for a good way of teaching networking concepts.

7.2 Future Work

A possible next step in this research could be to take a closer look at Containernet, providing an even more complete and detailed analysis of network emulation tools.

As written in previous chapters, Containernet is an extension of Mininet that implements Docker containers, making it an interesting compromise between Mininet's lightweight architecture and Kathara's more structured approach to managing network environments. One curious area worth exploring is how Containernet handles system resources. Since it builds on Mininet while integrating container management, it would be interesting to analyse how this affects performance, scalability, and overall resource efficiency. Comparing Containernet with both Mininet and Kathara could expose the trade-offs involved in container-based network emulation.

Another valuable direction would be examining the additional features Containernet provides beyond Mininet. A deeper dive into its APIs and available tools could reveal new capabilities that make it easier to set up, manage, or extend network environments. Understanding these differences could help determine when Containernet might be a better fit than traditional Mininet or Kathara.

A particularly interesting point of comparison would be how Containernet and Kathara handle filesystem management. Does Containernet, like Kathara, automatically configure certain services on virtual machines, or does it follow Mininet's approach of relying on the host system's built-in services? Investigating this aspect could clarify how these emulators differ in usability, setup complexity, and flexibility.

Finally, a more in-depth study of Containernet could focus on its real-world applications, exploring scenarios where it might offer a distinct advantage over Mininet or Kathara. This could help establish best practices for using Containernet in research and testing environments, broadening its potential for practical use.

Afterword

This thesis represents a significant learning experience in my academic journey. It marks my first true engagement with the research process, conducting experiments, analysing results, and synthesizing findings into a cohesive report.

Network emulation was unfamiliar territory when I began. My previous exposure was limited to using Packet Tracer during secondary education, which sparked my initial interest. The opportunity to explore professional emulators like Mininet and Kathara naturally appealed to me as an extension of that interest.

The research process began with literature review and preliminary experimentation, gradually building to independent creation and execution of network scenarios. However, this progression was not without challenges. Creating various scenarios presented numerous technical obstacles that required persistence to overcome. I needed to develop proficiency not only with the emulator interfaces but also with the underlying configuration files for both the emulators and the system itself. This troubleshooting process significantly enhanced my technical understanding of network emulation environments.

A critical phase of the research involved selecting appropriate benchmarking tools and designing meaningful test scenarios. The process of evaluating different methodologies and determining the most effective testing approaches provided valuable insights into systematic performance evaluation that extend beyond this specific research context.

I derived particular satisfaction from the comparative analysis—identifying differences between the emulators, documenting behavioural patterns under various conditions, and integrating these observations into a coherent analytical framework.

My investigation went beyond the initial scope, exploring additional network emulators and reviewing papers on advanced topics, revealing the technology's significance.

This thesis also represents a significant personal milestone, as I am the first member of my family to complete university education. This achievement carries special meaning and reflects not only personal effort but also the support received throughout my academic journey. I take this chance to thank again all the people who supported me through this journey

The field of networking has held my interest since secondary education, and this research has only reinforced that engagement. I view this thesis as a meaningful contribution to my academic development and am eager to further expand my knowledge in networking and information security domains that offer rich opportunities for continued professional and scholarly growth.

This work represents not merely the fulfilment of an academic requirement but a formative step in my ongoing intellectual development in computer science.

Bibliography

- [1] Broglio Matteo. Thesis material github repository. https://github.com/mbroglio/thesis_material. Accessed: July 7, 2025.
- [2] Lightyear Team. What is network emulation? <https://lightyear.ai/tips/what-is-network-emulation>, 2025. Accessed: 2025-02-25.
- [3] Bob Lantz and Brandon Heller. A network in a laptop: Rapid prototyping for software-defined networks. In *Hotnets-IX*, page 19, 10 2010.
- [4] Jose Gomez, Elie Kfoury, Jorge Crichigno, and Gautam Srivastava. A survey on network simulators, emulators, and testbeds used for research and education. *Computer Networks*, 237:110054, 10 2023.
- [5] iMedita. Top 10 list of network simulation tools. <https://www.imedita.com/blog/top-10-list-of-network-simulation-tools/>, 2024. Accessed: 2025-02-25.
- [6] Apposite Technologies. Network emulation vs. simulation: Which is right for you? <https://apposite-tech.com/network-emulation-vs-simulation/>, 2024. Accessed: 2025-02-25.
- [7] Ronny L. Bull, Rachel M. Dudukovich, Juan A. Fraire, Nadia Kortas, Robert Kassouf-Short, Aaron Smith, and Ethan Schweinsberg. Network emulation testbed capabilities for prototyping space dtn software and protocols. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–8, 2024.
- [8] Wenliang Du and Honghao Zeng. The seed internet emulator and its applications in cybersecurity education, 2022.
- [9] Keysight. Network emulation. <https://www.keysight.com/blogs/en/tech/educ/2024/network-emulation>, 2024. Accessed: 2025-02-25.
- [10] TechTarget. Network simulation vs. emulation: What’s the difference? <https://www.techtarget.com/searchnetworking/answer/Network-simulation-vs-emulation-Whats-the-difference>, 2024. Accessed: 2025-02-25.
- [11] iTrinegy. Network emulation essentials. https://www.networkology.com/download/partners/iTrinegy_Network_Emulation_Essentials-2020.pdf, 2020. Accessed: 2025-02-25.
- [12] Mininet Project. Mininet: An instant virtual network on your laptop (or other pc). <https://mininet.org/>. Accessed: 2025-02-25.

- [13] Mininet Developers. Mininet: A simple virtual network for experiments. <https://github.com/mininet/mininet>. Accessed: 2025-02-25.
- [14] Rogerio Oliveira, Christiane Schweitzer, Ailton Shinoda, and Ligia Prete. Using mininet for emulation and prototyping software-defined networks. In *2014 IEEE Colombian Conference on Communications and Computing (COLCOM)*, pages 1–6, 06 2014.
- [15] James T. Yu. Applying mininet for network education. In *Proceedings of the 53rd Annual Midwest Instruction and Computing Symposium (MICS 2020)*, pages 1–14. Curran Associates, Inc., 2020.
- [16] Ioannis Koulouras, Spiridoula V. Margariti, Ilias Bobotsaris, Eleftherios Stergiou, and Chrysostomos Stylios. On the performance of sdn controllers in real world topologies. In *2022 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pages 143–148, 2022.
- [17] University of South Carolina. Network tools and protocols lab 1: Introduction to mininet. https://research.cec.sc.edu/files/cyberinfra/files/NTP_Lab_Series.pdf, 2019. Accessed April 01, 2025.
- [18] G Florance and R J Anandhi. Study on sdn with security issues using mininet. *Recent Trends in Intensive Computing*, 2021.
- [19] Nader F. Mir, Jayashree Kotte, and Gokul Pokuri. Implementation of virtualization in software defined networking (sdn) for data center networks. In *2016 International Conference on Computational Science and Computational Intelligence (CSCI)*, pages 655–660, 2016.
- [20] Dong Yang and Wei-Tek Tsai. Sdn-based congestion control and bandwidth allocation scheme in 5g networks. *Sensors*, 24(3), 2024.
- [21] Javier Ortiz, Jorge Londoño, and Francisco Novillo. Evaluation of performance and scalability of mininet in scenarios with large data centers. In *2016 IEEE Ecuador Technical Chapters Meeting (ETCM)*, pages 1–6, 2016.
- [22] Kathará Development Team. Kathará: Lightweight container-based network emulation system. <https://www.kathara.org/>. Accessed: 2025-02-26.
- [23] Mariano Scazzariello, Lorenzo Ariemma, and Tommaso Caiazzzi. Kathará: A lightweight network emulation system. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, pages 1–2, 2020.
- [24] Gaetano Bonofiglio, Veronica Iovinella, Gabriele Lospoto, and Giuseppe Di Battista. Kathará: A container-based framework for implementing network function virtualization and software defined networks. *NOMS 2018 - 2018 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9, 2018.
- [25] Kathará Framework Contributors. Kathará framework on github. <https://github.com/KatharaFramework>. Accessed: 2025-02-26.
- [26] Netkit Team. Netkit: Network emulation for education. <https://www.netkit.org/>. Accessed: 2025-02-26.

- [27] Docker Inc. Docker: Accelerated container application development, 2025. Accessed: 2025-04-01.
- [28] Kathara Project. Megalos-(kathara-over-kubernetes). [https://github.com/KatharaFramework/Kathara/wiki/Megalos-\(Kathara-over-Kubernetes\)](https://github.com/KatharaFramework/Kathara/wiki/Megalos-(Kathara-over-Kubernetes)). Accessed: February 13, 2025.
- [29] Mariano Scazzariello, Lorenzo Ariemma, Giuseppe Di Battista, and Maurizio Patrignani. Megalos: A scalable architecture for the virtualization of large network scenarios. *Future Internet*, 13(9), 2021.
- [30] Mariano Scazzariello. Kathará: A lightweight and scalable network emulation system. In *RIPE 85 Meeting*, 2022. Accessed: 2025-02-26.
- [31] M. Großmann and D.T. Le. *Visualization of Network Emulation Enabled by Kathará*. Universität Würzburg, 2023.
- [32] Mininet Project. Mininet community extensions and packages. <https://github.com/mininet/mininet/wiki/Mininet-Community-Extensions-and-Packages>. Accessed: February 14, 2025.
- [33] Mininet-WiFi Project. Mininet-wifi: Emulator for software-defined wireless networks. <https://mininet-wifi.github.io/>. Accessed: April 05, 2025.
- [34] MaxiNet Project. Maxinet: Distributed emulation of software-defined networks. <https://maxinet.github.io/>. Accessed: April 05, 2025.
- [35] Manuel Peuster. Containernet: Use docker containers as hosts in mininet emulations. <https://containernet.github.io/>. Accessed: February 13, 2025.
- [36] M. Peuster, H. Karl, and S. van Rossem. Medicine: Rapid prototyping of production-ready network services in multi-pop environments. In *2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pages 148–153, Nov 2016.
- [37] Kathara Project. Kathara docker images. <https://github.com/KatharaFramework/Docker-Images>. Accessed: February 13, 2025.
- [38] Kathara Project. Kathará network plugin. <https://github.com/KatharaFramework/NetworkPlugin>. Accessed: April 05, 2025.
- [39] Kathara Project. Kathara external tools. <https://github.com/KatharaFramework/Kathara?tab=readme-ov-file#external-tools>. Accessed: April 05, 2025.
- [40] Mininet Project. Mininet: Download/get started. <https://mininet.org/download/>. Accessed: February 13, 2025.
- [41] Open vSwitch Project. Open vswitch. <https://www.openvswitch.org/>. Accessed: 2025-04-05.
- [42] The Linux Foundation. Opendaylight. <https://www.opendaylight.org/>. Accessed: 2025-04-05.

- [43] Ryu Community. Ryu sdn framework. <https://ryu-sdn.org/>. Accessed: 2025-04-05.
- [44] Brandon Heller. Ripl-pox. <https://github.com/brandonheller/riplpox>. Accessed: February 13, 2025.
- [45] Mininet Project. Mininet: Support. <https://mininet.org/support/>. Accessed: February 13, 2025.
- [46] IEEE. Ieee xplore digital library. <https://ieeexplore.ieee.org/Xplore/home.jsp>. Accessed: February 13, 2025.
- [47] Kathara Project. Kathara: Download & guides. <https://www.kathara.org/download.html>. Accessed: February 13, 2025.
- [48] Kathara Project. Kathara guides. <https://github.com/KatharaFramework/Kathara/wiki/Installation-Guides>. Accessed: February 13, 2025.
- [49] Kathara Project. Kathara: Success stories. <https://www.kathara.org/stories.html>. Accessed: February 13, 2025.